

BÀI TẬP LỚN

MÔN HỌC: ĐẶC TẢ PHẦN MỀM

HỆ THỐNG QUẢN LÝ BÁN HÀNG BILLING-APP

ĐẶC TẢ YÊU CẦU VÀ THIẾT KẾ

Mô hình miền, lớp chi tiết và cơ sở dữ liệu

Đối tượng khảo sát: Billing-app (React/Vite và Spring Boot)

Học phần: Đặc tả phần mềm

Phiên bản khảo sát: Mã nguồn hiện có tại ngày 24/09/2026

TÓM TẮT

Báo cáo đặc tả hệ thống Billing-app theo góc nhìn phân tích và thiết kế phần mềm. Nội dung được đối chiếu với mã nguồn hiện có, tài liệu tổng quan `CONTEXT.md`, các quyết định kiến trúc trong `docs/adr/` và migration Flyway `V1__init_schema.sql`. Trọng tâm là luồng xác thực phiên, lập đơn bán hàng, tích hợp thanh toán PayOS, bù trừ tồn kho bằng sổ cái giao dịch, kiểm kê và nhật ký hoạt động.

Báo cáo phân biệt ba mức chứng cứ: hành vi đang thể hiện trong mã nguồn; chủ trương được ghi trong ADR nhưng chưa thấy triển khai tương ứng; và thiết kế chuẩn hóa được đề xuất riêng để phục vụ phân tích học phần. Theo đó, lược đồ cơ sở dữ liệu đang triển khai không được gần nhãn đạt 3NF một cách tuyệt đối. Một ERD thực tế được trình bày cạnh một ERD 3NF đề xuất, đồng thời liệt kê rõ các ràng buộc chưa được khai báo trong migration.

Từ khóa: đặc tả phần mềm, POS, use case, mô hình miền, UML, Mermaid, ERD, Flyway, sổ cái tồn kho, PayOS.

Mục lục

TÓM TẮT	i
1 ĐẶC TẢ YÊU CẦU	1
1.1 Mục đích, cách tiếp cận và phạm vi hệ thống	1
1.2 Tác nhân và quyền hạn	2
1.3 Yêu cầu chức năng quan sát được	2
1.4 Yêu cầu chất lượng và ràng buộc vận hành	3
1.5 Đặc tả Use Case	3
1.5.1 UC-01 — Đăng nhập	5
1.5.2 UC-02 — Xoay refresh token	5
1.5.3 UC-03 — Quản lý danh mục, mặt hàng và biến thể	6
1.5.4 UC-04 — Tạo đơn tiền mặt	7
1.5.5 UC-05 — Tạo đơn QR PayOS và nhận webhook	8
1.5.6 UC-06 — Ghi giao dịch và kiểm kê kho	9
1.5.7 UC-07 — Hủy đơn PENDING	9
1.5.8 UC-08 — Chuyển QR sang tiền mặt	10
1.5.9 UC-09 — Xóa đơn	10
1.6 Truy vết yêu cầu — triển khai	10
2 ĐẶC TẢ THIẾT KẾ — MÔ HÌNH MIỀN	12
2.1 Phân rã các phân hệ miền	12
2.2 Sơ đồ Domain Model	12
2.3 Ý nghĩa thực thể và quan hệ	13
3 SƠ ĐỒ LỚP CHI TIẾT VÀ ĐẶC TẢ PHƯƠNG THỨC	15
3.1 Ranh giới mô hình lớp	15
3.2 Từ điển dữ liệu các lớp miền	18
3.2.1 Danh mục và cấu hình hàng bán	18
3.2.2 Bán hàng, khách hàng và khuyến mãi	20
3.2.3 Kho, định danh và kiểm toán	22
3.2.4 Các lớp điều phối trong sơ đồ	23
3.3 Đặc tả chi tiết các phương thức trọng tâm	23
3.3.1 OrderServiceImpl.createOrder	25
3.3.2 OrderServiceImpl.cancelOrder và compensatePendingOrder	26
3.3.3 OrderServiceImpl.deleteOrder	27
3.3.4 OrderServiceImpl.switchToCash	27

3.3.5	InventoryServiceImpl.recordTransaction	27
3.3.6	InventoryServiceImpl.performStockCheck	29
3.3.7	RefreshTokenService.rotate	29
3.4	Máy trạng thái đơn hàng và bất biến xuyên phân hệ	30
4	SƠ ĐỒ QUAN HỆ THỰC THỂ — ERD	32
4.1	ERD vật lý theo migration hiện hành	32
4.2	Danh mục khóa, nullability và ràng buộc thực tế	34
4.3	Đánh giá chuẩn hóa lược đồ hiện tại	36
4.4	ERD 3NF đề xuất — tách khỏi hiện trạng	36
4.4.1	Ràng buộc toàn vẹn nên khai báo trong thiết kế mục tiêu	39
4.4.2	ON DELETE/ON UPDATE cho quan hệ đề xuất	40
4.5	Ma trận khoảng cách giữa schema hiện tại và thiết kế 3NF	40
	KẾT LUẬN VÀ GIỚI HẠN ĐẶC TẢ	43

Danh mục hình

2.1	Domain Model Billing-app — quan hệ đối tượng và tham chiếu scalar	13
3.1	Lát cắt lớp chính theo lớp API, application, miền và hệ thống ngoài	15
4.1	ERD vật lý: chỉ thể hiện các khóa ngoại khai báo trong migration V1	32
4.2	ERD 3NF đề xuất — thiết kế mục tiêu, chưa triển khai	37

Danh mục bảng

1.1	Ranh giới hệ thống và các hệ thống bên ngoài	1
1.2	Danh mục tác nhân	2
1.3	Yêu cầu chức năng và căn cứ triển khai	2
1.4	Đặc tính chất lượng có bằng chứng trong mã nguồn	4
1.5	Liên kết yêu cầu với thành phần và bằng chứng	11
2.1	Từ điển khái niệm ở mức miền	14
3.10	Thuộc tính phụ thuộc và trách nhiệm của các lớp application/API	24
3.11	Hợp đồng phương thức createOrder	25
3.12	Hợp đồng hủy đơn và bù trừ	26
3.13	Hợp đồng xóa đơn	27
3.14	Hợp đồng chuyển PENDING QR sang tiền mặt	28
3.15	Hợp đồng ghi giao dịch tồn kho	28
3.16	Hợp đồng kiểm kê và điều chỉnh chênh lệch	29
3.17	Hợp đồng xoay refresh token	30
3.18	Chuyển trạng thái Order quan sát được	31
4.1	Khóa, unique index, FK và nullability đáng chú ý của schema thực tế	35
4.2	Danh mục NOT NULL/UNIQUE/CHECK đề xuất	39
4.3	Chính sách tham chiếu đề xuất	41
4.4	Các thay đổi dữ liệu cần có nếu chuyển từ V1 sang mô hình 3NF đề xuất	42

CHƯƠNG 1

ĐẶC TẢ YÊU CẦU

1.1 Mục đích, cách tiếp cận và phạm vi hệ thống

Billing-app là ứng dụng web hỗ trợ bán hàng tại quầy và quản trị dữ liệu cửa hàng. Mã nguồn được tổ chức thành hai phần: giao diện Front-end dùng React 19/Vite và API billingsoftware dùng Java/Spring Boot. Giao diện trao đổi với máy chủ qua REST/JSON; dữ liệu nghiệp vụ được lưu bằng MySQL qua Spring Data JPA, trong khi Flyway quản lý cấu trúc bảng. Các tích hợp quan sát được gồm PayOS cho liên kết/QR thanh toán và AWS S3 cho tài nguyên hình ảnh. Những thông tin này mô tả cấu hình và mã nguồn dự án, không tự thân chứng minh chất lượng vận hành hoặc kết quả nghiệm thu [8, 11, 9].

Phạm vi đặc tả tập trung vào hành vi hiện có: xác thực nhân viên, tra cứu và quản trị danh mục/sản phẩm, tạo đơn, áp dụng khuyến mãi, ghi nhận và điều chỉnh tồn kho, xử lý trạng thái đơn QR, cập nhật khách hàng tích lũy và xem nhật ký hoạt động. Báo cáo mô tả quy tắc từ controller, service, entity, repository, migration và ADR; yêu cầu phi chức năng chỉ được nêu khi có cấu hình hoặc cơ chế kiểm chứng được. Không suy diễn mục tiêu hiệu năng, SLA, quy mô dữ liệu hoặc quy trình ngoại tuyến từ tên sản phẩm hay tài liệu định hướng.

Hệ thống không có căn cứ cho thấy khách hàng cuối đăng nhập hoặc trực tiếp thao tác trên giao diện. Trong các luồng được khảo sát, nhân viên tạo đơn và có thể gán mã khách hàng/số điện thoại vào hồ sơ CRM. Vì vậy, “Khách hàng” được xem là thực thể nghiệp vụ được quản lý, không phải tác nhân tương tác trực tiếp. ADR 0002 mô tả khả năng offline-first, nhưng khảo sát mã giao diện không tìm thấy service worker, IndexedDB hay cơ chế đồng bộ offline hoàn chỉnh; tính năng đó không được xem là đã triển khai [10].

Bảng 1.1: Ranh giới hệ thống và các hệ thống bên ngoài

Thành phần	Vị trí so với ranh giới	Vai trò quan sát được
Front-end POS	Bên trong sản phẩm	Hiển thị màn hình đăng nhập, nghiệp vụ quản trị và bán hàng; gọi API qua Axios/REST.
Backend Billing-app	Bên trong sản phẩm	Áp dụng kiểm soát truy cập, xử lý nghiệp vụ, lưu dữ liệu và điều phối tích hợp.
MySQL	Phụ thuộc hạ tầng	Lưu entity nghiệp vụ, refresh token và dữ liệu nhật ký theo migration.
PayOS	Hệ thống bên ngoài	Tạo/hủy payment link và gửi webhook để thông báo kết quả thanh toán.
AWS S3	Hệ thống bên ngoài	Được cấu hình làm lưu trữ đối tượng/tệp; luồng cốt lõi của báo cáo không phụ thuộc thao tác S3.

Bảng 1.2: Danh mục tác nhân

Tác nhân	Phân loại	Trách nhiệm và giới hạn xác minh
Nhân viên (ROLE_USER)	Người dùng chính	Thực hiện thao tác bán hàng và truy cập các API nghiệp vụ được bảo vệ bằng <code>hasAnyRole("USER", "ADMIN")</code> . Trong UI có vai trò nhân viên; giá trị authority phải khớp cấu hình Spring Security.
Quản trị viên (ROLE_ADMIN)	Người dùng đặc quyền	Có thể truy cập API nghiệp vụ chung và các tuyến <code>/admin/**</code> ; các tuyến ghi giao dịch kho/kiểm kê được khai báo dưới tiền tố này.
PayOS	Hệ thống ngoài	Nhận yêu cầu tạo/hủy payment link; gọi <code>/payos/webhook</code> . API webhook tự xác minh chữ ký qua SDK PayOS.
MySQL	Dịch vụ hạ tầng	Nhận lệnh đọc/ghi do JPA/Flyway thực hiện; không phải tác nhân nghiệp vụ.
Khách hàng	Đối tượng miền	Được tra cứu qua mã hoặc số điện thoại để cập nhật chỉ số CRM; không có endpoint/giao diện khách hàng độc lập trong phạm vi xác minh.

1.2 Tác nhân và quyền hạn

Việc đặt tên “nhân viên” gắn với authority ROLE_USER là cách diễn giải vai trò sử dụng trong giao diện bán hàng. Cấu hình bảo mật kiểm tra vai trò kỹ thuật USER và ADMIN; báo cáo không giả định thêm mô hình phân quyền theo từng cửa hàng hoặc từng chi nhánh. Các tuyến công khai đáng chú ý gồm đăng nhập, refresh/logout và webhook PayOS; các API còn lại chịu quy tắc xác thực/ủy quyền tại SecurityConfig [13].

1.3 Yêu cầu chức năng quan sát được

Bảng 1.3: Yêu cầu chức năng và căn cứ triển khai

Mã	Yêu cầu	Tác nhân chính	Hành vi được quan sát
FR-01	Xác thực	Nhân viên, quản trị viên	Đăng nhập bằng email/mật khẩu; trả access token và đặt refresh token trong cookie HttpOnly.
FR-02	Duy trì phiên	Nhân viên, quản trị viên	Làm mới access token bằng refresh token; xoay refresh token và thu hồi cả family khi phát hiện token cũ bị phát lại.
FR-03	Quản trị danh mục/sản phẩm	Nhân viên, quản trị viên	Thao tác dữ liệu danh mục, mặt hàng, biến thể SKU và nhóm modifier qua các API quản trị.
FR-04	Tạo đơn bán hàng	Nhân viên, quản trị viên	Gửi các dòng hàng; máy chủ tính lại subtotal, discount, tax, grand total và lưu chi tiết đơn.

Mã	Yêu cầu	Tác nhân chính	Hành vi được quan sát
FR-05	Áp dụng khuyến mãi	Nhân viên, quản trị viên	Đánh giá promotion/coupon đủ điều kiện và chọn mức giảm tốt nhất; không cộng dồn nhiều promotion.
FR-06	Ghi nhận tồn kho	Quản trị viên	Ghi giao dịch IN/OUT/ADJUSTMENT; cập nhật cached stock theo tổng ledger.
FR-07	Kiểm kê biến thể	Quản trị viên	So sánh số lượng thực đếm với tồn hiện tại, ghi chênh lệch bằng ADJUSTMENT.
FR-08	Hủy đơn chờ	Nhân viên, quản trị viên	Chỉ chấp nhận PENDING; ghi bù trừ kho, hoàn lượt dùng promotion/CRM, yêu cầu hủy PayOS và chuyển trạng thái CANCELLED.
FR-09	Đổi QR sang tiền mặt	Nhân viên, quản trị viên	Chỉ chấp nhận PENDING; hủy link PayOS, đổi phương thức CASH và chuyển COMPLETED, giữ nguyên lần xuất kho ban đầu.
FR-10	Xóa đơn	Nhân viên, quản trị viên	Chỉ xóa PENDING hoặc CANCELLED; đơn PENDING phải được bù trừ trước, đơn COMPLETED bị từ chối.
FR-11	Nhận kết quả PayOS	PayOS	Xác minh webhook; nếu mã kết quả là 00 thì đánh dấu trạng thái thanh toán COMPLETED.
FR-12	Ghi nhật ký	Nhân viên, quản trị viên	Ghi sự kiện đăng nhập và thao tác nghiệp vụ; một số lỗi lưu activity log bị bắt và bỏ qua trong service.

1.4 Yêu cầu chất lượng và ràng buộc vận hành

Các mục dưới đây là đặc tính kiến trúc có căn cứ, không phải chỉ tiêu kiểm thử hiệu năng. Không tìm thấy mức đáp ứng tối đa, số giao dịch/giây, thời gian hoạt động cam kết hay yêu cầu RPO/RTO trong tài liệu đã khảo sát; vì vậy các giá trị đó được để ngoài phạm vi định lượng.

1.5 Đặc tả Use Case

Các tiền điều kiện nêu xác thực/quyền truy cập ở mức endpoint. Main Flow phản ánh luồng được triển khai phổ biến nhất; Alternative/Exception Flows ghi rõ nhánh rẽ có bằng chứng. Mã HTTP chính xác phụ thuộc exception handler của ứng dụng, do đó chỉ nêu trạng thái khi controller/service trực tiếp xác định được.

Bảng 1.4: Đặc tính chất lượng có bằng chứng trong mã nguồn

Mã	Thuộc tính	Ràng buộc/đặc tính quan sát được	Mức chứng cứ
QR-01	Bảo mật phiên	API dùng stateless session; access JWT và refresh token xoay vòng, refresh token lưu hash; cookie HttpOnly, SameSite và Secure cấu hình được.	Triển khai trong SecurityConfig, AuthController và RefreshTokenService.
QR-02	Phân quyền	/admin/** yêu cầu ADMIN; các tuyến nghiệp vụ được khai báo cho USER/ADMIN.	Khai báo trực tiếp trong SecurityConfig; không suy rộng sang từng thao tác ngoài cấu hình.
QR-03	Nhất quán tồn kho	Mỗi biến động được ghi thành ledger; số dư cached được tính lại từ giao dịch.	Thể hiện trong OrderServiceImpl và InventoryServiceImpl.
QR-04	Tích hợp thanh toán	Thông tin payment link/QR được lưu trong đơn; webhook xác minh chữ ký qua SDK.	Thể hiện trong OrderServiceImpl và PaymentWebhookController.
QR-05	Auditability	Nhiều thao tác ghi activity log; ActivityLogServiceImpl có nhánh bắt lỗi và không ném lại.	Nhật ký là nỗ lực best-effort, không thể kết luận mọi thay đổi đều có log bền vững.
QR-06	Ngoại tuyến	Không xác nhận được offline-first hoạt động từ mã nguồn hiện khảo sát.	ADR 0002 không thay thế bằng chứng service worker, IndexedDB hoặc đồng bộ thực tế.

1.5.1 UC-01 — Đăng nhập

Mã UC	UC-01 — Đăng nhập hệ thống
Tác nhân	Nhân viên (ROLE_USER), Quản trị viên (ROLE_ADMIN).
Tiền điều kiện	Tài khoản tồn tại và chưa bị vô hiệu hóa; request cung cấp email và mật khẩu. Endpoint POST /login công khai.
Luồng sự kiện chính	1. Người dùng gửi email/mật khẩu. 2. AuthController chuyển thông tin cho AuthenticationManager. 3. Khi thông tin hợp lệ, hệ thống tải UserDetails và phát access JWT. 4. RefreshTokenService tạo refresh token ngẫu nhiên, lưu SHA-256 hash cùng email, family và hạn dùng. 5. Hệ thống ghi sự kiện LOGIN, đặt raw refresh token trong cookie HttpOnly và trả AuthResponse gồm email, access token, role, name.
Luồng rẽ nhánh / ngoại lệ	Thông tin sai hoặc tài khoản không hoạt động: trả lỗi xác thực 401. Nếu ghi activity log thất bại, ActivityLogServiceImpl bắt lỗi và luồng đăng nhập vẫn có thể tiếp tục. Lỗi tạo token hoặc truy xuất người dùng làm request thất bại.
Hậu điều kiện	Người dùng nhận access token; trình duyệt giữ refresh token trong cookie với Path, SameSite, Secure và Max-Age theo cấu hình. Bản ghi token chỉ lưu hash.

1.5.2 UC-02 — Xoay refresh token

Mã UC	UC-02 — Làm mới phiên
Tác nhân	Nhân viên, quản trị viên; AuthController và RefreshTokenService xử lý bên trong.
Tiền điều kiện	Cookie refreshToken chứa raw token còn hiệu lực; endpoint POST /auth/refresh công khai nhưng yêu cầu token hợp lệ.
Luồng sự kiện chính	1. Backend bấm token nhận được và khóa bản ghi tương ứng khi tra cứu. 2. Nếu bản ghi còn hiệu lực, token cũ được thu hồi. 3. Cùng family được phát raw token mới và lưu hash/issuedAt/expiresAt. 4. AuthController tải lại user, phát JWT mới, cập nhật cookie và trả AuthResponse.
Luồng rẽ nhánh / ngoại lệ	Cookie thiếu/sai: xóa cookie và từ chối. Token hết hạn: đánh dấu revoked rồi từ chối. Token đã revoked được dùng lại: thu hồi mọi token còn hoạt động trong family, xóa cookie và từ chối.
Hậu điều kiện	Phiên dùng token mới; token cũ không còn hợp lệ. Khi phát hiện replay, family bị thu hồi để buộc xác thực lại.

1.5.3 UC-03 — Quản lý danh mục, mặt hàng và biến thể

Mã UC	UC-03 — Quản trị danh mục sản phẩm
Tác nhân	Nhân viên hoặc quản trị viên đã xác thực theo route tương ứng. AWS S3 là hệ thống ngoài trong luồng lưu ảnh.
Tiền điều kiện	Người dùng có quyền gọi API danh mục/mặt hàng; yêu cầu tạo/cập nhật đáp ứng kiểm tra dữ liệu của service.
Luồng sự kiện chính	1. Người dùng gửi yêu cầu tạo/sửa/xóa hoặc tra cứu Category/Item. 2. Controller chuyển request qua service, service kiểm tra định danh/quy tắc trùng lặp. 3. Item liên kết Category; các Variant giữ SKU, giá cơ sở, thuộc tính JSON và cached stock. 4. Item có thể liên kết ModifierGroup; mỗi nhóm có nhiều Modifier làm thay đổi giá hoặc tùy chọn chế biến. 5. Khi thao tác thay đổi dữ liệu thành công, service ghi activity log theo loại entity.
Luồng rẽ nhánh / ngoại lệ	Category không tồn tại, định danh/SKU trùng hoặc dữ liệu không hợp lệ: service trả lỗi nghiệp vụ. Ảnh được lưu qua cấu hình S3; lỗi tích hợp có thể khiến thao tác ảnh thất bại. Mã nguồn khảo sát không chứng minh một quy trình phân quyền khác nhau cho từng thao tác ngoài route policy.
Hậu điều kiện	Dữ liệu danh mục/Item/Variant/Modifier được đọc hoặc cập nhật. Quan hệ category-item và item-variant có khóa ngoại; liên kết item-modifier group là bảng nối.

1.5.4 UC-04 — Tạo đơn tiền mặt

Mã UC	UC-04 — Tạo và hoàn tất đơn CASH
Tác nhân	Nhân viên hoặc quản trị viên.
Tiền điều kiện	Người dùng đã xác thực; request gửi phương thức CASH và danh sách dòng hàng. Không có bước kiểm tra đủ tồn kho trước khi bán trong luồng createOrder.
Luồng sự kiện chính	1. Người dùng gửi OrderRequest gồm cartItems, thông tin khách hàng tùy chọn, coupon code và phương thức thanh toán. 2. Máy chủ chuyển giỏ hàng sang yêu cầu đánh giá promotion; tính lại subtotal, discount, tax và grand total. 3. Nếu tìm thấy Customer theo customerId hoặc phoneNumber, hệ thống chụp thông tin lên Order và tăng orderCount/totalSpent; nếu không, dùng giá trị khách mặc định. 4. Hệ thống tăng timesUsed cho promotion nếu còn quota. 5. Gán PaymentStatus COMPLETED cho CASH, ánh xạ OrderItem và lưu Order. 6. Với từng dòng có thể phân giải thành Variant, ghi giao dịch OUT âm và tính lại cached stock; ghi activity CREATE.
Luồng rẽ nhánh / ngoại lệ	Promotion hết quota sau đánh giá: request bị từ chối. Dòng hàng không ánh xạ được sang Variant thì không tạo giao dịch kho tương ứng. Tồn kho có thể xuống âm theo quy tắc hiện có. Request không có Customer hợp lệ vẫn tạo đơn với thông tin mặc định.
Hậu điều kiện	Đơn được lưu trạng thái COMPLETED; OrderItem được liên kết với Order. Số dư ledger giảm cho các Variant phân giải được, cached stock được tính lại. Chỉ số CRM/promotion được cập nhật nếu áp dụng.

1.5.5 UC-05 — Tạo đơn QR PayOS và nhận webhook

Mã UC	UC-05 — Thanh toán qua PayOS
Tác nhân	Nhân viên/quản trị viên khởi tạo; PayOS gọi webhook khi có kết quả.
Tiền điều kiện	Người dùng đã xác thực; OrderRequest có phương thức PAYOS; thông tin kết nối PayOS được cấu hình.
Luồng sự kiện chính	1. Backend áp dụng cùng quy trình tính tiền, cập nhật Customer/promotion và tạo OrderItem như UC-04. 2. PaymentStatus của đơn được đặt PENDING; các giao dịch OUT kho được ghi ngay khi tạo đơn. 3. Backend tạo PayOS payment link với orderCode từ khóa số của Order, tổng tiền, các item và URL trả về/hủy. 4. Backend lưu paymentLinkId, checkoutUrl, QR code, rồi trả OrderResponse. 5. PayOS POST webhook; backend xác minh signature qua SDK. Nếu mã giao dịch là 00, tra Order bằng orderCode và đặt trạng thái COMPLETED. PayOS tạo link lỗi: service ném lỗi runtime, giao dịch ứng dụng được đánh dấu rollback theo @Transactional nhưng kết quả từ hệ thống ngoài không thể suy ra chỉ từ rollback. Webhook chữ ký sai trả bad request; mã kết quả khác 00 trả bad request; orderCode không tìm thấy trả lỗi máy chủ. Không thấy kiểm tra idempotency riêng trong webhook.
Luồng rẽ nhánh / ngoại lệ	
Hậu điều kiện	Khi khởi tạo thành công: đơn PENDING, kho đã xuất và PayOS link được lưu. Khi webhook thành công: trạng thái chuyển COMPLETED. Tính duy nhất của webhook lặp không được đặc tả thêm trong mã nguồn khảo sát.

1.5.6 UC-06 — Ghi giao dịch và kiểm kê kho

Mã UC	UC-06 — Ghi nhận biến động tồn kho
Tác nhân	Quản trị viên đối với /admin/inventory/transactions và /admin/inventory/stock-check; người dùng đã xác thực có thể xem lịch sử theo biến thể.
Tiền điều kiện	Variant tồn tại. Giao dịch thường cần variantId, transactionType và quantity dương; kiểm kê cần actualCount không âm.
Luồng sự kiện chính	1. Người quản trị gửi yêu cầu nhập, xuất, điều chỉnh hoặc kiểm kê. 2. Với giao dịch thủ công, IN được chuẩn hóa thành trị tuyệt đối dương, OUT thành trị tuyệt đối âm; ADJUSTMENT nhận lượng đầu vào. 3. Với kiểm kê, hệ thống tính chênh lệch bằng actualCount trừ cached stock và ghi ADJUSTMENT; nếu trùng khớp vẫn ghi giao dịch có lượng 0. 4. Lưu giao dịch ledger, tổng hợp lại số lượng theo Variant và cập nhật cachedStockQuantity. 5. Trả InventoryTransactionResponse.
Luồng rẽ nhánh / ngoại lệ	Thiếu variantId/type, quantity không dương hoặc actualCount âm: lỗi 400. Không tìm thấy Variant: lỗi 404. Dữ liệu tồn kho không bị chặn ở ngưỡng 0; giao dịch kiểm kê có thể tạo điều chỉnh âm/dương.
Hậu điều kiện	Ledger có bản ghi mới và cache số lượng được tái tính. GET lịch sử trả danh sách giao dịch theo Variant/thời gian.

1.5.7 UC-07 — Hủy đơn PENDING

Mã UC	UC-07 — Hủy đơn chờ xử lý
Tác nhân	Nhân viên hoặc quản trị viên; PayOS là bên được yêu cầu hủy link khi phương thức là PAYOS.
Tiền điều kiện	Order tồn tại và PaymentStatus là PENDING.
Luồng sự kiện chính	1. Người dùng gọi POST /orders/{orderId}/cancel. 2. Backend kiểm tra tồn tại và trạng thái PENDING. 3. Đặt trạng thái CANCELLED và chạy compensatePendingOrder. 4. Với các dòng hàng phân giải được Variant, ghi ledger IN theo số lượng tuyệt đối và tái tính cache. 5. Giảm timesUsed nếu Promotion còn tồn tại; gọi Customer.revertOrderSpending nếu Customer còn tồn tại. 6. Với PAYOS, yêu cầu hủy payment link; lưu Order, ghi CANCEL và trả OrderResponse.
Luồng rẽ nhánh / ngoại lệ	Order không tồn tại: 404. Trạng thái khác PENDING hoặc thiếu PaymentDetails: 400. Lỗi gọi PayOS khi hủy bị bắt và ghi cảnh báo, không ngăn cập nhật trạng thái cục bộ; vì vậy thành công của trạng thái local không khẳng định link từ xa đã hủy.
Hậu điều kiện	Đơn chuyển CANCELLED; giao dịch bù trừ và chỉ số CRM/promotion được cập nhật nếu các đối tượng liên quan còn tồn tại.

1.5.8 UC-08 — Chuyển QR sang tiền mặt

Mã UC	UC-08 — Chuyển phương thức sang CASH
Tác nhân	Nhân viên hoặc quản trị viên; PayOS được gọi để hủy link nếu đơn hiện là PAYOS.
Tiền điều kiện	Order tồn tại, có PaymentDetails và trạng thái PENDING.
Luồng sự kiện chính	1. Người dùng gọi POST /orders/{orderId}/switch-to-cash. 2. Backend kiểm tra trạng thái PENDING. 3. Nếu đang dùng PAYOS, gọi thao tác hủy link từ xa. 4. Đổi paymentMethod thành CASH và PaymentStatus thành COMPLETED. 5. Lưu Order, ghi UPDATE activity log và trả OrderResponse.
Luồng rẽ nhánh / ngoại lệ	Đơn không tồn tại: 404. Đơn không PENDING hoặc không có PaymentDetails: 400. Lỗi PayOS bị bắt trong helper; số lượng kho không ghi OUT lần hai và cũng không được hoàn IN ở nhánh đổi tiền mặt.
Hậu điều kiện	Đơn ở trạng thái COMPLETED/CASH; lần xuất kho và cập nhật CRM/promotion khi tạo đơn vẫn được giữ nguyên.

1.5.9 UC-09 — Xóa đơn

Mã UC	UC-09 — Xóa đơn chưa hoàn tất
Tác nhân	Nhân viên hoặc quản trị viên.
Tiền điều kiện	Order tồn tại. Chỉ PENDING hoặc CANCELLED được phép xóa.
Luồng sự kiện chính	1. Người dùng gọi DELETE /orders/{orderId}. 2. Backend tải đơn và đọc trạng thái. 3. Với PENDING, chạy bù trừ kho, promotion, CRM và PayOS trước khi xóa. 4. Với CANCELLED, bỏ qua bù trừ để tránh hoàn lặp. 5. Xóa Order, cascade xóa OrderItem theo quan hệ hiện có và ghi log DELETE.
Luồng rẽ nhánh / ngoại lệ	Không tìm thấy đơn: 404. COMPLETED hoặc trạng thái ngoài PENDING/CANCELLED: 400. Việc xóa đã hủy không tạo thêm giao dịch bù trừ.
Hậu điều kiện	Order bị xóa khỏi bảng; lịch sử giao dịch kho và activity log lưu tách biệt. Việc bảo toàn dữ liệu kế toán cho đơn hoàn tất được thực hiện ở service, không phải bằng trigger/CHECK trong schema.

1.6 Truy vết yêu cầu — triển khai

Bảng 1.5: Liên kết yêu cầu với thành phần và bằng chứng

Yêu cầu	Thành phần chính	Bằng chứng mã nguồn	Mức đặc tả
FR-01/02	AuthController, RefreshTokenService	controller/AuthController.java; service/RefreshTokenService.java	Triển khai; lỗi xác thực có nhánh rõ.
FR-04, 05, 08, 09, 10	OrderController, OrderServiceImpl, PromotionServiceImpl	controller/OrderController.java; service/impl/OrderServiceImpl.java; service/impl/PromotionServiceImpl.java	Triển khai; một số snapshot là scalar.
FR-06/07	InventoryController, InventoryServiceImpl	controller/InventoryController.java; service/impl/InventoryServiceImpl.java	Triển khai; signed quantity là invariant nghiệp vụ.
FR-11	PaymentWebhookController	controller/PaymentWebhookController.java	Triển khai xác minh webhook; chưa thấy idempotency guard.
FR-12	ActivityLogServiceImpl	service/impl/ActivityLogServiceImpl.java	Best-effort; lỗi lưu bị bắt.
QR-06	Front-end worker/storage	Đã tìm kiếm service worker, IndexedDB và cờ trạng thái mạng trong cây giao diện khảo sát	ADR định hướng, chưa xác minh triển khai.

CHƯƠNG 2

ĐẶC TẢ THIẾT KẾ — MÔ HÌNH MIỀN

2.1 Phân rã các phân hệ miền

Mô hình nghiệp vụ có thể phân thành năm phân hệ liên kết. **Danh mục hàng hóa** tổ chức Category, Item và Variant; Item là sản phẩm cha, Variant (SKU) là đơn vị có giá cơ sở, thuộc tính biến thể JSON và số lượng tồn. **Tùy chọn bán hàng** tổ chức ModifierGroup / Modifier độc lập với Variant; modifier làm thay đổi giá hoặc thông tin chế biến và không được coi là mặt hàng tồn kho. Quan hệ nhiều-nhiều giữa Item và ModifierGroup cho phép một nhóm lựa chọn áp dụng cho nhiều Item.

Bán hàng và thanh toán xoay quanh Order và OrderItem. Order lưu tổng tiền, thông tin khách hàng/khuyến mãi dạng snapshot, phương thức thanh toán và PaymentDetails được nhúng. Mỗi OrderItem giữ mã hàng/biến thể dạng chuỗi cùng tên và giá tại thời điểm bán; vì thế chi tiết lịch sử không phải một liên kết JPA tới Item/Variant. PayOS được điều phối ở service và webhook cập nhật trạng thái.

Kho lấy InventoryTransaction làm sổ cái biến động. IN ghi lượng dương, OUT ghi lượng âm, ADJUSTMENT lưu chênh lệch có thể âm, dương hoặc bằng không. Cached stock trên Variant được tái tính từ tổng giao dịch. Do hệ thống chủ động cho phép tồn âm, cached stock không thể bị ràng buộc không âm nếu vẫn giữ quy tắc bán hiện tại.

Khuyến mãi và quan hệ khách hàng quản lý điều kiện Promotion, mã coupon, giới hạn sử dụng và chỉ số tích lũy của Customer. Khi tạo đơn, hệ thống tìm khách theo mã rồi theo điện thoại; đơn giữ ID/tên snapshot chứ không có khóa ngoại tới Customer. Promotion cũng được lưu trên Order dưới dạng mã/tên snapshot. Cuối cùng, **định danh và kiểm toán** dùng User, RefreshToken và ActivityLog để quản lý phiên và ghi lịch sử, nhưng RefreshToken/ActivityLog liên kết tài khoản qua email dạng scalar, không bằng FK trong migration.

2.2 Sơ đồ Domain Model

Hình dưới đây thể hiện quan hệ khái niệm quan sát được. Nét liền biểu diễn quan hệ entity/JPA hoặc bảng nối; nét đứt diễn tả tham chiếu nghiệp vụ bằng mã scalar, không hàm ý FK. Sơ đồ Mermaid được dùng làm biểu diễn văn bản có thể lưu phiên bản, đối chiếu và tái sinh; ký hiệu UML tham chiếu đặc tả UML 2.5.1 [2], còn cú pháp class diagram tham chiếu tài liệu Mermaid [3, 4].

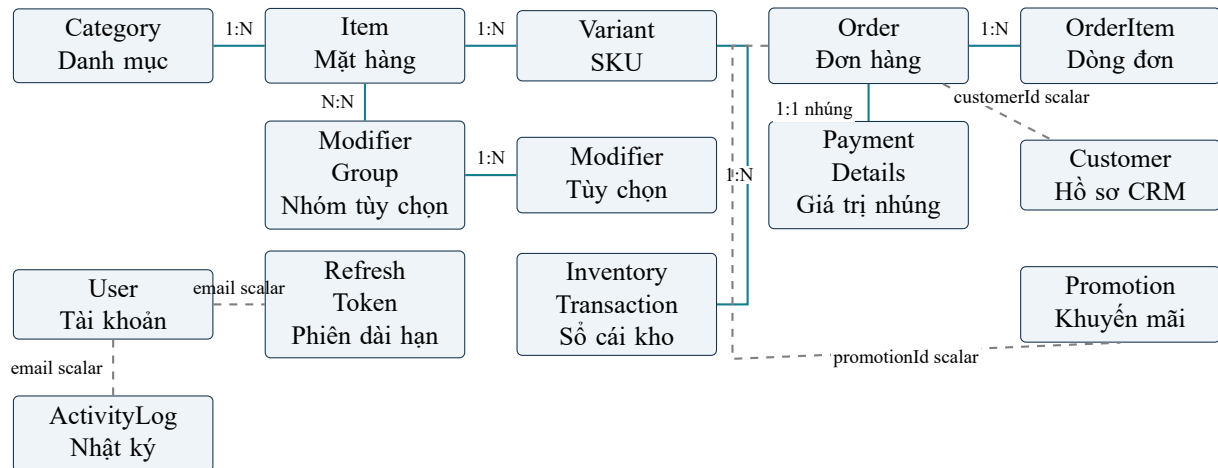
Mã Mermaid tương ứng:

Listing 2.1: Domain Model dưới dạng Mermaid classDiagram

```

1 classDiagram
2   Category "1" --> "0..*" Item : category
3   Item "1" --> "0..*" Variant : variants
4   Item "0..*" --> "0..*" ModifierGroup : modifierGroups / join table
5   ModifierGroup "1" --> "0..*" Modifier : modifiers
6   Order "1" --> "0..*" OrderItem : items
7   Order "1" --> "0..1" PaymentDetails : embedded value
8   Variant "1" --> "0..*" InventoryTransaction : ledger

```



Hình 2.1: Domain Model Billing-app — quan hệ đối tượng và tham chiếu scalar

```

9  Order ..> Customer : customerId lookup, no FK
10 Order ..> Promotion : promotionId snapshot, no FK
11 User ..> RefreshToken : userEmail scalar, no FK
12 User ..> ActivityLog : userEmail scalar, no FK

```

2.3 Ý nghĩa thực thể và quan hệ

Multiplicity phải được hiểu ở đúng tầng mô hình. Trong lược đồ vật lý, Item–ModifierGroup được cài đặt thành hai quan hệ 1:N qua `tbl_item_modifier_groups`, tạo quan hệ khái niệm N:N. Customer–Order và Promotion–Order có thể mang ý nghĩa lookup trong nghiệp vụ, nhưng schema chỉ lưu business ID/snapshot nên quan hệ không được DB bảo đảm. Tương tự, OrderItem.itemId/variantId không có FK. Mọi sơ đồ triển khai phía sau giữ ranh giới này để không nhầm liên kết logic với ràng buộc tham chiếu thật.

Bảng 2.1: Từ điển khái niệm ở mức miền

Khái niệm	Trách nhiệm	Quan hệ/đặc tính nghiệp vụ
Category	Nhóm phân loại hàng hóa.	Một Category có thể chứa nhiều Item; Item giữ FK category ở lược đồ triển khai.
Item	Sản phẩm cha có mô tả, hình ảnh, Category và tập biến thể.	Một Item có nhiều Variant; có thể nối nhiều ModifierGroup qua bảng trung gian.
Variant	Đơn vị hàng có SKU, và tồn cache.	Là đơn vị ghi nhận tồn; liên kết InventoryTransaction thuộc tính động, basePrice 1:N.
ModifierGroup / Modifier	Mô hình tùy chọn thêm, ví dụ nhóm topping/độ ngọt.	Modifier thuộc một nhóm; nhóm có min/max selections; không mang số lượng tồn.
Order/OrderItem	Hóa đơn giao dịch và các dòng bán tại thời điểm tạo.	Một Order có các OrderItem; dòng lưu snapshot item/variant/giá/tên; FK hiện có chỉ là orderId nội bộ.
PaymentDetails	Trạng thái, payment link, checkout URL và QR data.	Là @Embeddable, được nhúng vào tbl_orders; không phải bảng thanh toán riêng.
Inventory Transaction	Sự kiện bất biến tương đối mô tả biến động hàng.	Variant cha; quantity có dấu; cached stock được tính từ tổng ledger.
Customer/Promotion	Dữ liệu CRM và cấu hình điều kiện giảm giá.	Được tìm/cập nhật bằng business ID; order chỉ lưu mã/tên snapshot, không có JPA relation.
User / RefreshToken / ActivityLog	Tài khoản, token xoay vòng và dấu vết thao tác.	Liên kết phiên/nhật ký bằng email scalar trong schema hiện tại.

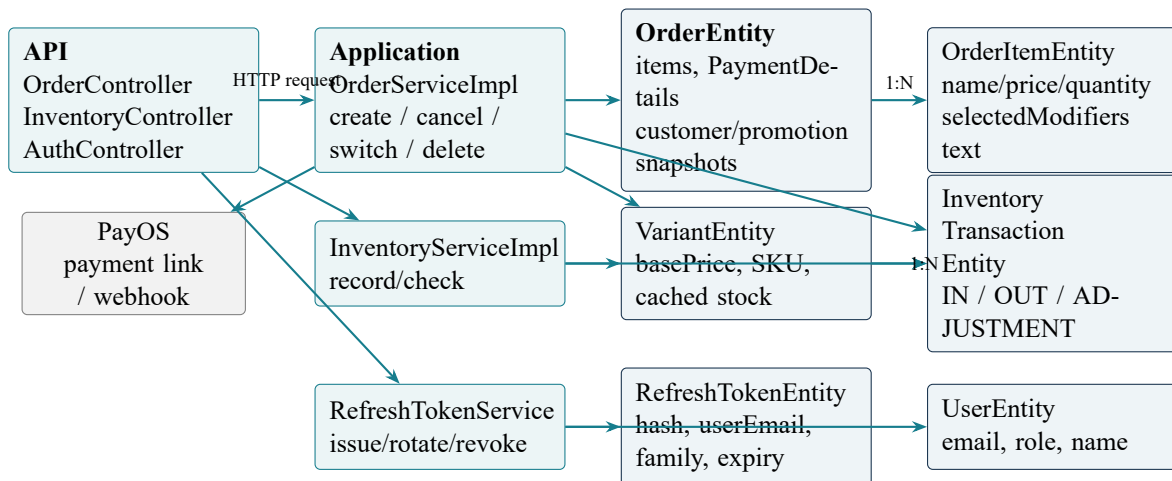
CHƯƠNG 3

SƠ ĐỒ LỚP CHI TIẾT VÀ ĐẶC TẢ PHƯƠNG THỨC

3.1 Ranh giới mô hình lớp

Sơ đồ lớp chi tiết chọn lát cắt xuyên suốt bốn nghiệp vụ có nhiều bất biến nhất: đơn hàng/thanh toán, sổ cái tồn kho, phiên refresh token, và các thực thể danh mục/khách hàng/khuyến mãi hỗ trợ. Controller thể hiện điểm vào HTTP; service điều phối nghiệp vụ; entity biểu diễn trạng thái lưu trữ. Các DTO chỉ xuất hiện như kiểu tham số/kiểu trả về, còn chi tiết cấu hình framework, mapper, filter và mọi CRUD phụ trợ lặp lại được lược khỏi hình để giữ sơ đồ đọc được. Vì thế sơ đồ không được hiểu là bản kiểm kê mọi lớp Java của repository.

Trong sơ đồ Mermaid, ký hiệu - là thành viên private, + là public; > biểu diễn phụ thuộc gọi service. Các quan hệ liên giữa entity phản ánh mapping JPA hiện có. Những ID customer/promotion/order-line hoặc email actor được giữ như scalar không được nâng thành association JPA. Các getter/setter sinh bởi Lombok không liệt kê; bảng thuộc tính mô tả visibility khai báo trong source.



Hình 3.1: Lát cắt lớp chính theo lớp API, application, miền và hệ thống ngoài

Mã Mermaid đầy đủ cho lát cắt lớp:

Listing 3.1: Sơ đồ lớp chi tiết ở dạng Mermaid classDiagram

```

classDiagram
class OrderController {
    -orderService: OrderService
    +createOrder(request: OrderRequest): OrderResponse
    +cancelOrder(orderId: String): OrderResponse
    +switchToCash(orderId: String): OrderResponse
    +deleteOrder(orderId: String): void
}
class OrderServiceImpl {
    -orderEntityRepository: OrderEntityRepository
    -payOS: PayOS
    -variantRepository: VariantRepository
    -inventoryTransactionRepository: InventoryTransactionRepository
}

```

```

14  -promotionService: PromotionService
15  -promotionRepository: PromotionRepository
16  -customerRepository: CustomerRepository
17  -activityLogService: ActivityLogService
18  +createOrder(request: OrderRequest): OrderResponse
19  +cancelOrder(orderId: String): OrderResponse
20  +switchToCash(orderId: String): OrderResponse
21  +deleteOrder(orderId: String): void
22  -compensatePendingOrder(order: OrderEntity, note: String, reason: String): void
23  }
24  class InventoryController {
25      -inventoryService: InventoryService
26      +recordTransaction(request: InventoryTransactionRequest):
        InventoryTransactionResponse
27      +performStockCheck(request: StockCheckRequest): InventoryTransactionResponse
28  }
29  class InventoryServiceImpl {
30      +recordTransaction(request: InventoryTransactionRequest):
        InventoryTransactionResponse
31      +performStockCheck(request: StockCheckRequest): InventoryTransactionResponse
32  }
33  class AuthController {
34      -refreshTokenService: RefreshTokenService
35      +login(request: AuthRequest): AuthResponse
36      +refresh(rawRefreshToken: String): AuthResponse
37      +logout(rawRefreshToken: String): void
38  }
39  class RefreshTokenService {
40      -refreshTokenRepository: RefreshTokenRepository
41      -refreshTokenExpirationMs: long
42      +issue(userEmail: String): IssuedRefreshToken
43      +rotate(rawToken: String): IssuedRefreshToken
44      +revoke(rawToken: String): void
45  }
46  class OrderEntity {
47      -id: Long
48      -orderId: String
49      -customerId: String
50      -subtotal: Double
51      -grandTotal: Double
52      -items: List~OrderItemEntity~
53      -paymentDetails: PaymentDetails
54      -paymentMethod: PaymentMethod
55      +addOrderItem(item: OrderItemEntity): void
56  }
57  class OrderItemEntity {
58      -id: Long
59      -itemId: String
60      -variantId: String
61      -quantity: Integer
62      -selectedModifiers: String
63      -order: OrderEntity
64  }
65  class PaymentDetails {
66      -orderId: String
67      -status: PaymentStatus
68      -paymentLinkId: String
69      -checkoutUrl: String
70      -qrCode: String

```

```
71 }
72 class VariantEntity {
73     -variantId: String
74     -sku: String
75     -basePrice: BigDecimal
76     -attributes: Map~String,String~
77     -cachedStockQuantity: Integer
78 }
79 class InventoryTransactionEntity {
80     -transactionId: String
81     -variant: VariantEntity
82     -transactionType: TransactionType
83     -quantity: Integer
84     -referenceId: String
85 }
86 class RefreshTokenEntity {
87     -tokenHash: String
88     -userEmail: String
89     -familyId: String
90     -issuedAt: Timestamp
91     -expiresAt: Timestamp
92     -revokedAt: Timestamp
93 }
94 class UserEntity {
95     -userId: String
96     -email: String
97     -password: String
98     -role: String
99     -name: String
100 }
101 class CustomerEntity {
102     -customerId: String
103     -name: String
104     -phoneNumber: String
105     -totalSpent: Double
106     -orderCount: Integer
107     +revertOrderSpending(orderTotal: Double): void
108 }
109 class PromotionEntity {
110     -promotionId: String
111     -type: PromotionType
112     -discountType: DiscountType
113     -discountValue: BigDecimal
114     -usageLimit: Integer
115     -timesUsed: Integer
116 }
117 class ActivityLogEntity {
118     -logId: String
119     -userEmail: String
120     -action: String
121     -entityType: String
122     -entityId: String
123     -createdAt: LocalDateTime
124 }
125 class CategoryEntity
126 class ItemEntity
127 class ModifierGroupEntity
128 class ModifierEntity
129 class PaymentWebhookController
```

```

130 class PayOS
131   OrderController ..> OrderServiceImpl
132   InventoryController ..> InventoryServiceImpl
133   AuthController ..> RefreshTokenService
134   OrderServiceImpl --> OrderEntity
135   OrderServiceImpl ..> CustomerEntity : lookup by customerId / phone
136   OrderServiceImpl ..> PromotionEntity : lookup by promotionId
137   OrderEntity "1" *-- "0..*" OrderItemEntity
138   OrderEntity *-- PaymentDetails : embedded
139   VariantEntity "1" <-- "0..*" InventoryTransactionEntity
140   ItemEntity "1" --> "0..*" VariantEntity
141   CategoryEntity "1" --> "0..*" ItemEntity
142   ItemEntity "0..*" --> "0..*" ModifierGroupEntity : join table
143   ModifierGroupEntity "1" --> "0..*" ModifierEntity
144   RefreshTokenService --> RefreshTokenEntity
145   UserEntity .. RefreshTokenEntity : scalar email, no FK
146   UserEntity .. ActivityLogEntity : scalar email, no FK
147   PaymentWebhookController ..> PayOS
148   PaymentWebhookController ..> OrderEntity

```

3.2 Từ điển dữ liệu các lớp miền

Các bảng sau ghi kiểu dữ liệu Java và visibility khai báo. Cột quan hệ mang kiểu entity/danh sách thay cho kiểu SQL. Mọi trường được liệt kê là private trong mã nguồn; Lombok sinh accessor tùy annotation, còn các thao tác public quan trọng được phân tích ở mục 3.4. Giá trị mặc định được ghi khi có ý nghĩa nghiệp vụ.

3.2.1 Danh mục và cấu hình hàng bán

CategoryEntity — danh mục

Thuộc tính	Kiểu Java	Phạm vi	Ý nghĩa
id	Long	private	Khóa nội bộ tự tăng.
categoryId	String	private	Mã danh mục nghiệp vụ; có unique index nhưng migration cho phép NULL.
name	String	private	Tên danh mục; unique index ở DB, nullable theo migration.
description	String	private	Mô tả.
bgColor	String	private	Mã/mô tả màu nền giao diện.
imgUrl	String	private	URL ảnh danh mục.
createdAt	Timestamp	private	Thời điểm tạo.
updatedAt	Timestamp	private	Thời điểm cập nhật.

ItemEntity — mặt hàng cha

Thuộc tính	Kiểu Java	Phạm vi	Ý nghĩa
id	Long	private	Khóa nội bộ.
itemId	String	private	Mã mặt hàng nghiệp vụ, unique ở migration nhưng cho phép NULL trong schema.
name	String	private	Tên hiển thị.
description	String	private	Nội dung mô tả.
price	BigDecimal	private	Giá ở cấp Item; trường hiện diện cùng giá basePrice của Variant.
createdAt / updatedAt	Timestamp	private	Mốc tạo/cập nhật.
imgUrl	String	private	URL ảnh.
category	CategoryEntity	private	Quan hệ ManyToOne tới Category.
variants	List<VariantEntity>	private	Tập biến thể; quan hệ OneToMany, cascade theo mapping.
modifierGroups	List<ModifierGroupEntity>	private	Tập nhóm modifier; quan hệ ManyToMany qua bảng nối.

VariantEntity — biến thể/SKU

Thuộc tính	Kiểu Java	Phạm vi	Ý nghĩa
id	Long	private	Khóa nội bộ.
variantId	String	private	Mã biến thể duy nhất.
sku	String	private	Mã SKU duy nhất.
basePrice	BigDecimal	private	Giá cơ sở; được dùng để chụp giá dòng hàng.
attributes	Map<String, String>	private	Thuộc tính biến thể động; converter lưu ở cột TEXT.
cachedStockQuantity	Integer	private	Bản cache tổng ledger; có thể âm theo quy tắc chấp nhận bán khi thiếu hàng.
item	ItemEntity	private	Quan hệ ManyToOne tới Item.
createdAt / updatedAt	Timestamp	private	Mốc tạo/cập nhật.

ModifierGroupEntity và ModifierEntity — nhóm/tùy chọn thêm

Lớp.thuộc tính	Kiểu Java	Phạm vi	Ý nghĩa
Group.id	Long	private	Khóa nội bộ nhóm.
Group.groupId	String	private	Mã nhóm duy nhất.
Group.name / description	String	private	Tên bắt buộc trong migration và mô tả tùy chọn.
Group.minSelections / maxSelections	Integer	private	Số lựa chọn tối thiểu/tối đa; mặc định lần lượt 0 và 1.

Lớp.thuộc tính	Kiểu Java	Phạm vi	Ý nghĩa
Group.modifiers	List<ModifierEntity>	private	Tập tùy chọn thuộc nhóm.
Group.items	List<ItemEntity>	private	Tập Item liên kết qua bảng nối.
Group.createdAt / updatedAt	Timestamp	private	Mốc tạo/cập nhật.
Modifier.id	Long	private	Khóa nội bộ tùy chọn.
Modifier.modifierId	String	private	Mã modifier duy nhất.
Modifier.name	String	private	Tên tùy chọn.
Modifier.priceAdjustment	BigDecimal	private	Phần cộng/trừ giá, mặc định BigDecimal.ZERO.
Modifier.modifierGroup	ModifierGroupEntity	private	Quan hệ ManyToOne về nhóm.
Modifier.createdAt / updatedAt	Timestamp	private	Mốc tạo/cập nhật.

3.2.2 Bán hàng, khách hàng và khuyến mãi

OrderEntity — đơn hàng

Thuộc tính	Kiểu Java	Phạm vi	Ý nghĩa
id	Long	private	PK số; đồng thời dùng làm orderCode khi tạo yêu cầu PayOS.
orderId	String	private	Mã đơn nghiệp vụ ORD . . . ; cột order_code unique NOT NULL.
customerId / customerName / phoneNumber	String	private	Mã và snapshot khách hàng; không có relation/FK tới Customer.
subtotal / discountAmount / tax / grandTotal	Double	private	Tổng tiền; discountAmount mặc định 0.0.
promotionId / promotionName	String	private	Mã/tên promotion snapshot.
createdAt	LocalDateTime	private	Thời điểm tạo; gán ở callback PrePersist.
items	List<OrderItemEntity>	private	Danh sách dòng; OneToMany cascade ALL, orphanRemoval=true.
paymentDetails	PaymentDetails	private	Giá trị embeddable gồm trạng thái, link và QR.
paymentMethod	PaymentMethod	private	Enum CASH hoặc PAYOS; lưu dạng chuỗi.

OrderItemEntity và PaymentDetails — dòng đơn và trạng thái thanh toán

Lớp.thuộc tính	Kiểu Java	Phạm vi	Ý nghĩa
OrderItem.id	Long	private	Khóa dòng tự tăng.

Lớp.thuộc tính	Kiểu Java	Phạm vi	Ý nghĩa
OrderItem.itemId / variantId	String	private	Mã mặt hàng/biến thể dạng snapshot scalar; không có FK.
OrderItem.name	String	private	Tên hàng tại thời điểm bán.
OrderItem.basePrice / price	Double	private	Giá cơ sở và giá dòng tại thời điểm bán.
OrderItem.quantity	Integer	private	Số lượng dòng; cột migration nullable, không có CHECK.
OrderItem.selectedModifiers	String	private	Nội dung tùy chọn được lưu vào cột TEXT.
OrderItem.order	OrderEntity	private	ManyToOne tới đơn, cột order_id là FK.
Payment.orderId	String	private	Mã PayOS/domain được đặt bởi service.
Payment.status	PaymentStatus	private	PENDING / COMPLETED / FAILED / CANCELLED; cột migration TINYINT ordinal.
Payment.paymentLinkId	String	private	Định danh payment link.
Payment.checkoutUrl	String	private	URL thanh toán, độ dài cột 1000.
Payment.qrCode	String	private	Dữ liệu QR, cột TEXT.

CustomerEntity và PromotionEntity — CRM/khuyến mãi

Lớp.thuộc tính	Kiểu Java	Phạm vi	Ý nghĩa
Customer.id	Long	private	Khóa nội bộ.
Customer.customerId	String	private	Mã khách hàng duy nhất, NOT NULL trong migration.
Customer.name / phoneNumber	String	private	Tên và điện thoại; phoneNumber unique, NOT NULL.
Customer.email	String	private	Email tùy chọn.
Customer.totalSpent	Double	private	Tổng tiền tích lũy; mặc định 0.0, cập nhật khi tạo/hủy đơn.
Customer.orderCount	Integer	private	Số đơn tích lũy; mặc định 0.
Customer.createdAt / updatedAt	Timestamp	private	Mốc tạo/cập nhật tự động.
Promotion.id / promotionId	Long / String	private	Khóa nội bộ và mã promotion duy nhất.
Promotion.name / description	String	private	Tên bắt buộc và mô tả tùy chọn.
Promotion.type / discountType	PromotionType / DiscountType	private	Phân loại promotion và cách tính giảm.
Promotion.code	String	private	Coupon code tùy chọn, unique index ở DB nhưng cho phép NULL.

Lớp.thuộc tính	Kiểu Java	Phạm vi	Ý nghĩa
Promotion.discountValue / maxDiscountAmount	BigDecimal	private	Giá trị giảm và trần giảm tùy chọn.
Promotion.minOrderAmount	BigDecimal	private	Giá trị đơn tối thiểu, mặc định ZERO.
Promotion.startDate / endDate	LocalDate	private	Khoảng ngày hiệu lực.
Promotion.startTime / endTime	LocalTime	private	Khoảng giờ hiệu lực.
Promotion.daysOfWeek	String	private	Chuỗi ngày áp dụng, không phải bảng nói ngày.
Promotion.buyVariantId / getVariantId	String	private	Mã biến thể cho khuyến mãi dạng mua/tặng.
Promotion.bogoDiscountPercent	BigDecimal	private	Phần trăm giảm ở nhánh BOGO, mặc định 100.
Promotion.usageLimit / timesUsed	Integer	private	Giới hạn và bộ đếm lượt dùng.
Promotion.isActive	Boolean	private	Cờ kích hoạt, mặc định true.
Promotion.createdAt / updatedAt	Timestamp	private	Mốc tạo/cập nhật.

3.2.3 Kho, định danh và kiểm toán

InventoryTransactionEntity và RefreshTokenEntity

Lớp.thuộc tính	Kiểu Java	Phạm vi	Ý nghĩa
InventoryTx.id	Long	private	PK nội bộ.
InventoryTx.transactionId	String	private	Mã transaction duy nhất, NOT NULL.
InventoryTx.variant	VariantEntity	private	ManyToOne, FK variant_id bắt buộc; ON DELETE CASCADE.
InventoryTx.transactionType	TransactionType	private	IN, OUT hoặc ADJUSTMENT, lưu enum dạng String.
InventoryTx.quantity	Integer	private	Lượng có dấu: IN dương, OUT âm, ADJUSTMENT chênh lệch.
InventoryTx.referenceId / note	String	private	Tham chiếu nghiệp vụ và ghi chú.
InventoryTx.createdAt	Timestamp	private	Thời điểm tạo, Hibernate gán.
RefreshToken.id	Long	private	PK nội bộ tự tăng.
RefreshToken.tokenHash	String	private	SHA-256 digest Base64 URL-safe; unique NOT NULL.

Lớp.thuộc tính	Kiểu Java	Phạm vi	Ý nghĩa
RefreshToken. userEmail	String	private	Email tài khoản; NOT NULL nhưng migration không tạo FK user.
RefreshToken. familyId	String	private	Định danh family; có index tra cứu.
RefreshToken. issuedAt / expiresAt	Timestamp	private	Thời điểm phát hành và hết hạn.
RefreshToken. revokedAt	Timestamp	private	Thời điểm thu hồi; NULL nghĩa là chưa bị thu hồi.

UserEntity và ActivityLogEntity

Lớp.thuộc tính	Kiểu Java	Phạm vi	Ý nghĩa
User.id	Long	private	PK nội bộ.
User.userId / email	String	private	Mã tài khoản và email; unique index trong migration.
User.password	String	private	Chuỗi xác thực đã qua encoder khi tạo/lưu tài khoản.
User.role	String	private	Authority dùng cho phân quyền.
User.name	String	private	Tên hiển thị trong AuthResponse/UI.
User.createdAt / updatedAt	Timestamp	private	Mốc tạo/cập nhật.
ActivityLog.id	Long	private	PK nội bộ.
ActivityLog.logId	String	private	Mã nhật ký unique NOT NULL.
ActivityLog. userEmail	String	private	Email người thao tác, NOT NULL, không có FK.
ActivityLog.action	String	private	Hành động như LOGIN / CREATE / UPDATE / DELETE / CANCEL.
ActivityLog. entityType / entityId	String	private	Loại đối tượng và business ID liên quan.
ActivityLog. description	String	private	Mô tả tóm tắt, tối đa 1000 ký tự theo migration.
ActivityLog. createdAt	LocalDateTime	private	Thời điểm ghi log, NOT NULL.

3.2.4 Các lớp điều phối trong sơ đồ

3.3 Đặc tả chi tiết các phương thức trọng tâm

Các hợp đồng dưới đây mô tả chữ ký và hiệu ứng trực tiếp của lớp service. Tiên điều kiện được chia thành điều kiện do route/service kiểm tra và điều kiện môi trường. Nếu mã nguồn không kiểm tra một điều kiện, báo cáo ghi “không được kiểm tra tường minh” thay vì biến nó thành yêu cầu đã triển khai. Transactional annotation chỉ bảo đảm phạm vi giao dịch cơ sở dữ liệu trong ứng dụng; không làm cho thao tác với PayOS trở thành giao dịch phân tán.

Bảng 3.10: Thuộc tính phụ thuộc và trách nhiệm của các lớp application/API

Lớp	Thuộc tính khai báo (đều private)	Phương thức công khai trọng tâm
OrderController	OrderService orderService	GET danh sách/phân trang; POST tạo; DELETE; GET latest/by id; POST cancel/switch-to-cash.
OrderServiceImpl	repository Order; PayOS; VariantRepository; InventoryTransactionRepository; PromotionService/Repository; CustomerRepository; ActivityLogService; ObjectMapper.	createOrder, cancelOrder, switchToCash, deleteOrder, getOrdersPaginated.
InventoryController	InventoryService inventoryService	POST ghi giao dịch/kiểm kê; GET toàn bộ và theo Variant.
InventoryServiceImpl	InventoryTransactionRepository, VariantRepository	recordTransaction, performStockCheck, getTransactionsByVariant.
AuthController	PasswordEncoder, AuthenticationManager, AppUserDetailsService, UserService, JwtUtil, ActivityLogService, RefreshTokenService; cấu hình cookie/TTL.	login, refresh, logout; helper authenticate và cookie private.
RefreshTokenService	RefreshTokenRepository, SecureRandom, refreshTokenExpirationMs	issue, rotate, revoke; hash và phát raw token là helper private.
PaymentWebhookController	PayOS, OrderEntityRepository	handleWebhook(Webhook): xác minh callback, cập nhật trạng thái thành COMPLETED khi code=00.
Repository ports	Không có state instance nghiệp vụ riêng trong interface; khai báo truy vấn Spring Data/JPA.	Tra cứu theo mã/email, lưu entity, khóa token khi rotate, tổng hợp stock ledger.

3.3.1 OrderServiceImpl.createOrder

Bảng 3.11: Hợp đồng phương thức createOrder

Thành phần	Đặc tả
Chữ ký	<code>createOrder(request: OrderRequest): OrderResponse; public, @Transactional.</code>
Input	OrderRequest gồm customerId/customerName/phoneNumber, cartItems, couponCode, appliedPromotionId và paymentMethod. Mỗi OrderItemRequest gồm itemId, variantId, name, basePrice, price, quantity, selectedModifiers. Các tổng client gửi không phải nguồn cuối: service thay bằng kết quả evaluatePromotion.
Output	OrderResponse đã ánh xạ từ OrderEntity mới lưu, gồm trạng thái, tổng tiền, dòng đơn và PaymentDetails khi có.
Tiền điều kiện	Request đã được JSON deserialize; người gọi đã qua security filter. Service không có kiểm tra tường minh giỏ hàng khác null/không rỗng, số lượng dương, giá client hợp lệ hoặc đủ tồn kho tại entry point. Hợp đồng bên trong phụ thuộc PromotionService.evaluatePromotion và kiểu paymentMethod hợp lệ.
Hậu điều kiện	Lưu Order cùng các dòng; cập nhật số liệu khách hàng/promotion nếu tìm thấy; ghi OUT cho Variant phân giải được; tái tính cached stock; với PAYOS tạo và lưu payment link. Với CASH trạng thái ban đầu là COMPLETED, còn phương thức khác được đặt PENDING theo nhánh hiện tại.
Lỗi/ngoại lệ	Đạt quota promotion giữa lúc đánh giá và lúc ghi nhận: HTTP 400. Lỗi PayOS: RuntimeException. Lỗi DB/ánh xạ: transaction ứng dụng thất bại. Không tìm được Variant cho dòng không làm toàn bộ request bị từ chối trong phần xử lý stock quan sát được.

Thuật toán quan sát được:

```
function createOrder(request):
    cartForEvaluation = map request.cartItems to EvaluationCartItem
    evaluation = promotionService.evaluatePromotion(couponCode,
    cartForEvaluation)
    order = map request to OrderEntity
    order.subtotal      = evaluation.subtotal
    order.discountAmount = evaluation.discountAmount
    order.tax           = evaluation.tax
    order.grandTotal    = evaluation.grandTotal
    order.promotion     = evaluation.appliedPromotion

    customer = find by trimmed customerId
    if customer is absent and phone is nonblank and phone != "0000000000":
        customer = find by trimmed phone
    if customer exists:
        copy customer snapshot to order
        increment customer.orderCount and totalSpent by grandTotal
        persist customer
    else:
        set fallback customer name/phone when absent
```

```

if an applied promotion exists:
    reload promotion
    if usageLimit reached: reject with BAD_REQUEST
    increment timesUsed and persist

set payment status: CASH -> COMPLETED, otherwise -> PENDING
map cart items; attach each item to order; persist order
for each saved line:
    variant = find by variantId, else first variant for itemId
    if variant exists:
        append OUT transaction with quantity = -abs(line.quantity or 1)
        cachedStock = sum(ledger quantities for variant)
if payment method is PAYOS:
    create payment link using order database id as orderCode
    save link id, checkout URL and QR data
attempt CREATE activity log
return mapped OrderResponse

```

Một hệ quả nghiệp vụ quan trọng là sổ kho được cập nhật khi tạo đơn, trước khi webhook báo thanh toán thành công. Đơn PENDING do đó đã giữ chỗ bằng cách trừ tồn. Nhánh hủy phải bù IN; nhánh đổi sang CASH không trừ lần nữa. Không có bước kiểm tra số dư tồn trước OUT, phù hợp với quy tắc cho phép tồn âm trong ADR 0002 nhưng không chứng minh ứng dụng thực hiện đồng bộ offline.

3.3.2 OrderServiceImpl.cancelOrder và compensatePendingOrder

Bảng 3.12: Hợp đồng hủy đơn và bù trừ

Thành phần	Đặc tả
Chữ ký public	cancelOrder(orderId: String): OrderResponse; public, transactional.
Chữ ký helper	compensatePendingOrder(order: OrderEntity, ledgerNotePrefix: String, payosCancelReason: String): void; private.
Input	Mã đơn nghiệp vụ. Helper nhận entity đã nạp cùng nội dung ghi chú và lý do hủy liên kết PayOS.
Output	OrderResponse ở trạng thái CANCELLED; helper không trả giá trị.
Tiền điều kiện	Tìm thấy đơn; PaymentDetails khác null; PaymentStatus đúng PENDING.
Hậu điều kiện	Trạng thái local thành CANCELLED; ghi IN cho từng Variant tìm được; cache kho tính lại; hoàn bộ đếm promotion nếu lớn hơn 0; gọi Customer.revertOrderSpending khi Customer còn tồn tại; yêu cầu PayOS hủy link; lưu đơn và log CANCEL.
Lỗi/ngoại lệ	Không tìm thấy đơn: 404. Không có PaymentDetails/PENDING: 400. Lỗi remote PayOS bị catch và chỉ ghi warning, nên hậu điều kiện “PayOS đã hủy” không được bảo đảm.

```

function cancelOrder(orderId):
    order = findByOrderId(orderId) or NOT_FOUND
    if order.paymentDetails is absent or status != PENDING:
        reject BAD_REQUEST
    order.paymentDetails.status = CANCELLED

```

```

    compensatePendingOrder(order, cancellation note, PayOS reason)
    save order
    attempt CANCEL activity log
    return order response

function compensatePendingOrder(order, notePrefix, reason):
    for each line in order.items:
        variant = find by line.variantId, else first variant for line.itemId
        if variant exists:
            append IN transaction with quantity = abs(line.quantity or 1)
            recalculate variant cached stock from ledger
    if order has promotionId:
        if promotion exists and timesUsed > 0: decrement and save
    if order has customerId and customer exists:
        revert customer orderCount/totalSpent and save
    if order.paymentMethod == PAYOS:
        try cancel remote payment link; catch exception and log warning

```

3.3.3 OrderServiceImpl.deleteOrder

Bảng 3.13: Hợp đồng xóa đơn

Thành phần	Đặc tả
Chữ ký	deleteOrder(orderId: String): void; public, transactional; controller trả HTTP 204 nếu không có lỗi.
Input / output	orderId dạng String; không có body kết quả.
Tiền điều kiện	Order tồn tại; PaymentDetails có thể null nhưng trạng thái null/khác PENDING-CANCELLED sẽ bị từ chối.
Hậu điều kiện	PENDING: bù trừ trước khi xóa. CANCELLED: không bù lần hai. Xóa Order dẫn đến xóa OrderItem bằng cascade JPA/ON DELETE CASCADE; log DELETE được gọi sau thao tác xóa.
Lỗi/ngoại lệ	Không tìm thấy: 404. COMPLETED: HTTP 400 với thông điệp bảo vệ lịch sử kế toán. FAILED/null/trạng thái khác: HTTP 400. Lỗi từ DB làm transaction thất bại.

Đây là invariant được cưỡng chế trong service, không phải constraint ở tầng bảng. Một lệnh SQL trực tiếp ngoài application vẫn có thể xóa đơn COMPLETED nếu tài khoản DB cho phép. Nhận định này phân biệt quy tắc ứng dụng với bất biến vật lý.

3.3.4 OrderServiceImpl.switchToCash

3.3.5 InventoryServiceImpl.recordTransaction

```

function recordTransaction(request):
    require nonblank request.variantId
    require request.quantity > 0
    require request.transactionType exists
    variant = findVariant(request.variantId) or NOT_FOUND
    if type == IN: signedQuantity = abs(quantity)

```

Bảng 3.14: Hợp đồng chuyển PENDING QR sang tiền mặt

Thành phần	Đặc tả
Chữ ký	<code>switchToCash(orderId: String): OrderResponse</code> ; public, transactional.
Tiền điều kiện	Order tồn tại, có <code>PaymentDetails</code> và đang PENDING.
Hậu điều kiện	Nếu method cũ là PAYOS thì gọi hủy link; đổi <code>paymentMethod</code> CASH, status COMPLETED; lưu và log UPDATE. Không ghi giao dịch kho mới.
Ngoại lệ	Không tìm thấy: 404; trạng thái hoặc <code>payment details</code> không đúng: 400. Helper bắt lỗi hủy PayOS và tiếp tục cập nhật local.
Ràng buộc	Giữ số lượng đã trừ, promotion và CRM đã ghi khi tạo đơn; tránh hoàn rồi trừ lại làm phát sinh bù trừ lặp. Kết quả remote cancel không được xác nhận trong response.

Bảng 3.15: Hợp đồng ghi giao dịch tồn kho

Thành phần	Đặc tả
Chữ ký	<code>recordTransaction(request: InventoryTransactionRequest): InventoryTransactionResponse</code> ; public, transactional.
Input	<code>variantId</code> : String; <code>quantity</code> : Integer; <code>transactionType</code> : TransactionType; <code>referenceId/note</code> tùy chọn.
Output	Response gồm transaction đã lưu và cached stock mới.
Tiền điều kiện	<code>variantId</code> không trống; <code>quantity</code> khác null và lớn hơn 0; <code>transactionType</code> khác null; Variant tồn tại.
Hậu điều kiện	IN lượng dương; OUT lượng âm theo <code>-abs(quantity)</code> ; ADJUSTMENT giữ <code>quantity</code> gửi vào. Lưu/flush ledger, sum ledger theo Variant, cập nhật <code>cachedStockQuantity</code> .
Ngoại lệ	Thiếu ID/type hoặc <code>quantity</code> không hợp lệ: 400. Không thấy Variant: 404.

```

else if type == OUT: signedQuantity = -abs(quantity)
else: signedQuantity = quantity
tx = saveAndFlush(variant, type, signedQuantity, referenceId, note)
stock = sumLedger(variant.variantId)
variant.cachedStockQuantity = stock or 0
save variant
return response(tx, variant.cachedStockQuantity)

```

Với API ghi giao dịch thủ công, quantity đầu vào bắt buộc lớn hơn 0 nên ADJUSTMENT trực tiếp qua route này không thể giảm tồn bằng số âm. API kiểm kê ở mục sau tạo discrepancy âm từ actualCount-currentStock. Đây là khác biệt giữa hai đường ghi dữ liệu; không nên gộp cả hai thành một quy tắc đầu vào duy nhất.

3.3.6 InventoryServiceImpl.performStockCheck

Bảng 3.16: Hợp đồng kiểm kê và điều chỉnh chênh lệch

Thành phần	Đặc tả
Chữ ký	performStockCheck(request: StockCheckRequest): InventoryTransactionResponse; public, transactional.
Input	variantId: String; actualCount: Integer; referenceId/note tùy chọn.
Output	Bản ghi ADJUSTMENT và cached stock sau khi tái tính.
Tiền điều kiện	variantId không trống; actualCount khác null và không âm; Variant tồn tại.
Hậu điều kiện	discrepancy = actualCount - cachedStock (null coi như 0); ghi ADJUSTMENT với lượng discrepancy. Nếu không truyền ref, sinh CHECK-...; nếu không có note thì tạo ghi chú khớp/chênh lệch. Lưu và tính lại ledger/cache.
Ngoại lệ	ID thiếu hoặc actualCount âm: 400. Variant không tồn tại: 404.
Ràng buộc	discrepancy có thể âm hoặc bằng 0; không được áp ràng buộc quantity > 0 cho bảng ledger nếu muốn bảo toàn kiểm kê/xuất kho.

3.3.7 RefreshTokenService.rotate

```

function rotate(rawToken):
  if rawToken is null or blank: reject INVALID_TOKEN
  existing = findByHashForUpdate(SHA256(rawToken)) or INVALID_TOKEN
  now = currentTimestamp()
  if existing.revokedAt is not null:
    request revocation of every active token in existing.familyId
    reject INVALID_TOKEN
  if existing.expiresAt <= now:
    existing.revokedAt = now
    save existing
    reject INVALID_TOKEN
  existing.revokedAt = now
  save existing
  return issue(existing.userEmail, existing.familyId)

```

Bảng 3.17: Hợp đồng xoay refresh token

Thành phần	Đặc tả
Chữ ký	<code>rotate(rawToken: String): IssuedRefreshToken</code> ; public, transactional.
Input / output	Raw token từ cookie; trả token mới, hạn dùng và userEmail cho AuthController. Raw token mới chỉ trả qua object nội bộ/cookie; DB lưu digest.
Tiền điều kiện	Token không null/trống; SHA-256 hash trùng tokenHash trong DB; bản ghi chưa revoke và expiresAt còn sau thời điểm hiện tại.
Hậu điều kiện dự kiến	Thu hồi token cũ và phát token mới trong cùng family, lưu hash/hạn mới.
Lỗi/ngoại lệ	Thiếu/sai/hết hạn/revoked: <code>ResponseStatusException</code> không xác thực được. Replay gọi <code>revokeActiveByFamilyId</code> rồi ném lỗi. Vì method transactional và lỗi là unchecked, theo quy tắc rollback mặc định của Spring, cần xác minh thao tác thu hồi family có được commit hay bị rollback; mã nguồn được khảo sát không cho thấy <code>noRollbackFor</code> hoặc transaction riêng [7].
Ràng buộc	Hành vi “thu hồi cả family sau replay” là ý định thể hiện bằng lệnh repository; không báo cáo như kết quả bền vững đã được kiểm chứng.

Tài liệu Spring mô tả rollback mặc định cho unchecked exceptions trong giao dịch khai báo [7]. Do nhánh replay vừa thay đổi family vừa ném unchecked exception trong cùng phương thức transactional, đây là điểm cần xử lý/kiểm chứng riêng nếu yêu cầu bảo mật bắt buộc thu hồi family phải được bảo đảm. Báo cáo không biến “đã gọi câu lệnh UPDATE” thành “đã commit thành công”.

3.4 Máy trạng thái đơn hàng và bất biến xuyên phân hệ

Một số bất biến nghiệp vụ có thể phát biểu bằng ngôn ngữ đặc tả như sau: (i) tổng tiền dùng trong đơn phải lấy từ kết quả tính tại máy chủ; (ii) mỗi lần hủy PENDING chỉ được bù trừ một lần; (iii) chuyển QR sang CASH không phát sinh giao dịch kho OUT lần hai; (iv) completed order không xóa qua service; (v) cached stock bằng tổng signed quantity của ledger tại thời điểm tái tính; (vi) các quan hệ ID dạng snapshot không được xem là FK; (vii) PayOS và DB là hai ranh giới giao dịch khác nhau; (viii) ghi activity log hiện là best-effort vì lỗi ghi bị hấp thụ trong service.

Bảng 3.18: Chuyển trạng thái Order quan sát được

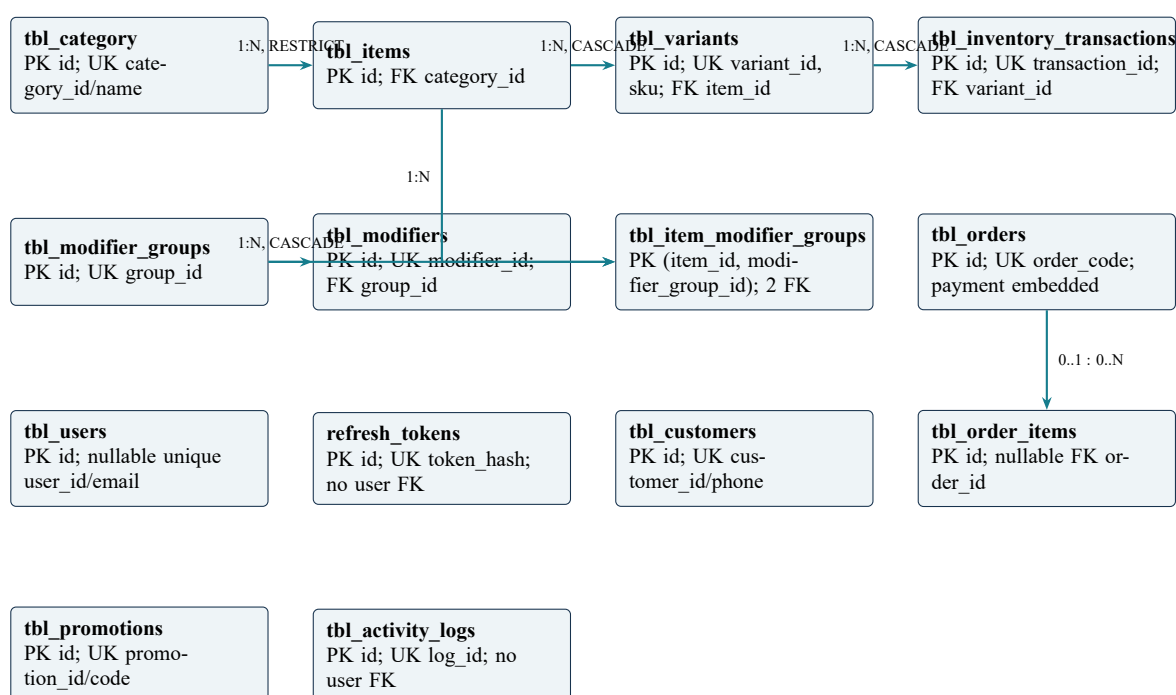
Trạng thái ban đầu	Sự kiện	Trạng thái kế tiếp	Hiệu ứng kèm theo
Mới tạo, CASH	createOrder	COMPLETED	Ledger OUT đã ghi; CRM/promotion cập nhật; không tạo PayOS link.
Mới tạo, PAYOS	createOrder	PENDING	Ledger OUT đã ghi; PayOS link/QR lưu sau khi tạo link.
PENDING	webhook code=00	COMPLETED	Cập nhật status thanh toán; không thấy kiểm tra transition/idempotency tường minh.
PENDING	cancelOrder	CANCELLED	Ledger IN bù trừ; hoàn promotion/CRM; thử hủy PayOS.
PENDING/PAYOS	switchToCash	COMPLETED/CASH	Thử hủy link, giữ lần OUT ban đầu.
PENDING	deleteOrder	Đã xóa	Bù trừ trước khi xóa.
CANCELLED	deleteOrder	Đã xóa	Không bù trừ lặp.
COMPLETED	deleteOrder	Không đổi	Bị từ chối tại service.

CHƯƠNG 4

SƠ ĐỒ QUAN HỆ THỰC THỂ — ERD

4.1 ERD vật lý theo migration hiện hành

Migration V1__init_schema.sql tạo **14 bảng** InnoDB. Sơ đồ sau phản ánh FK được khai báo thật; nó không bổ sung đường nối chỉ vì hai bảng chứa mã có tên tương tự. Một số bảng như tbl_orders, tbl_customers, tbl_promotions, refresh_tokens và tbl_activity_logs không có FK nối tới entity liên quan. PaymentDetails được nhúng vào tbl_orders, do đó chưa có quan hệ bảng 1:1 riêng trong ERD thực tế.



Hình 4.1: ERD vật lý: chỉ thể hiện các khóa ngoại khai báo trong migration V1

Mã Mermaid erDiagram — crow's foot cho schema thực tế:

Listing 4.1: ERD hiện thực theo 14 bảng trong migration V1

```

1 erDiagram
2     TBL_USERS {
3         bigint id PK
4         varchar user_id UK "nullable"
5         varchar email UK "nullable"
6         varchar role "nullable"
7     }
8     REFRESH_TOKENS {
9         bigint id PK
10        varchar token_hash UK
11        varchar user_email
12        varchar family_id
13        timestamp expires_at
14        timestamp revoked at "nullable"

```

```
15 }
16 TBL_CATEGORY {
17     bigint id PK
18     varchar category_id UK "nullable"
19     varchar name UK "nullable"
20 }
21 TBL_ITEMS {
22     bigint id PK
23     varchar item_id UK "nullable"
24     bigint category_id FK
25     decimal price "nullable"
26 }
27 TBL_VARIANTS {
28     bigint id PK
29     varchar variant_id UK
30     varchar sku UK
31     decimal base_price
32     text attributes "nullable"
33     int cached_stock_quantity
34     bigint item_id FK
35 }
36 TBL_MODIFIER_GROUPS {
37     bigint id PK
38     varchar group_id UK
39     int min_selections
40     int max_selections
41 }
42 TBL_MODIFIERS {
43     bigint id PK
44     varchar modifier_id UK
45     decimal price_adjustment
46     bigint group_id FK
47 }
48 TBL_ITEM_MODIFIER_GROUPS {
49     bigint item_id PK, FK
50     bigint modifier_group_id PK, FK
51 }
52 TBL_CUSTOMERS {
53     bigint id PK
54     varchar customer_id UK
55     varchar phone_number UK
56     double total_spent
57     int order_count
58 }
59 TBL_PROMOTIONS {
60     bigint id PK
61     varchar promotion_id UK
62     varchar code UK "nullable"
63     varchar days_of_week "nullable"
64     int times_used
65 }
66 TBL_ORDERS {
67     bigint id PK
68     varchar order_code UK
69     varchar customer_id "snapshot, no FK"
70     varchar promotion_id "snapshot, no FK"
71     tinyint status "ordinal"
72     varchar payment_method
73 }
```

```

74  TBL_ORDER_ITEMS {
75    bigint id PK
76    bigint order_id FK "nullable"
77    varchar item_id "no FK"
78    varchar variant_id "no FK"
79    text selected_modifiers "nullable"
80  }
81  TBL_INVENTORY_TRANSACTIONS {
82    bigint id PK
83    varchar transaction_id UK
84    bigint variant_id FK
85    varchar transaction_type
86    int quantity
87  }
88  TBL_ACTIVITY_LOGS {
89    bigint id PK
90    varchar log_id UK
91    varchar user_email
92    varchar action
93    varchar entity_id "polymorphic scalar"
94  }
95  TBL_CATEGORY ||--o{ TBL_ITEMS : contains
96  TBL_ITEMS ||--o{ TBL_VARIANTS : defines
97  TBL_ITEMS ||--o{ TBL_ITEM_MODIFIER_GROUPS : links
98  TBL_MODIFIER_GROUPS ||--o{ TBL_ITEM_MODIFIER_GROUPS : links
99  TBL_MODIFIER_GROUPS ||--o{ TBL_MODIFIERS : contains
100 TBL_ORDERS o|--o{ TBL_ORDER_ITEMS : has_nullable_fk_lines
101 TBL_VARIANTS ||--o{ TBL_INVENTORY_TRANSACTIONS : ledger

```

Mermaid marker `||--o{` biểu diễn một parent với không hoặc nhiều dòng con; marker `o|--o{` được dùng cho Order–OrderItem vì cột `order_id` trong migration cho phép NULL. Liên kết Item–ModifierGroup là N:N ở mức khái niệm, nhưng trong ERD vật lý được triển khai qua `tbl_item_modifier_groups`. Mermaid hỗ trợ các ký hiệu này trong ER diagram [4]. Sơ đồ có 1:N và N:N qua bảng nối; 1:1 hiện chưa được lưu bằng bảng riêng. Bảng `tbl_users`, `refresh_tokens`, `tbl_customers`, `tbl_promotions` và `tbl_activity_logs` không được nối tới nhau ở hình thực tế khi thiếu FK.

4.2 Danh mục khóa, nullability và ràng buộc thực tế

Tất cả bảng dùng khóa nội bộ `id BIGINT AUTO_INCREMENT` làm PK, ngoại trừ bảng nối có PK ghép (`item_id`, `modifier_group_id`). Cột business ID có unique index nhằm nhận diện tài nguyên. Tuy nhiên, unique index trên cột cho phép NULL không đồng nghĩa với candidate key theo nghĩa quan hệ: MySQL cho phép nhiều NULL ở unique index; vì vậy các khóa business nullable trong migration được ghi là “unique index nullable”, không được gọi là candidate key tuyệt đối. Quy tắc CHECK và FK được diễn giải theo tài liệu MySQL [5, 6].

Bảng 4.1: Khóa, unique index, FK và nullability đáng chú ý của schema thực tế

Bảng	PK	Candidate/unique key	FK và hành động xóa	NOT NULL ngoài PK
tbl_users	id	user_id, email (unique nhưng nullable)	Không có	Không có trong migration
refresh_tokens	id	token_hash	Không có FK tới User	token_hash, user_email, family_id, issued_at, expires_at
tbl_category	id	category_id, name (unique nhưng nullable)	Không có	Không có
tbl_items	id	item_id (unique nhưng nullable)	category_id → category.id, RESTRICT	category_id
tbl_variants	id	variant_id, sku	item_id → items.id, CASCADE	variant_id, sku, base_price, cached_stock_quantity, item_id
tbl_modifier_groups	id	group_id	Không có	group_id, name, min_selections, max_selections
tbl_modifiers	id	modifier_id	group_id → modifier_groups.id, CASCADE	modifier_id, name, price_adjustment, group_id
tbl_item_modifier_groups	cột ghép	(item_id, modifier_group_id)	item/group → cha, CASCADE	item_id, modifier_group_id
tbl_customers	id	customer_id, phone_number	Không có	customer_id, name, phone_number, total_spent, order_count
tbl_promotions	id	promotion_id; code unique nullable	Không có	promotion_id, name, type, discount_value, min_order_amount, times_used, is_active
tbl_orders	id	order_code	Không có FK tới Customer/Promotion	order_code
tbl_order_items	id	Không có unique business key	order_id → orders.id, CASCADE; cột nullable	Không có
tbl_inventory_transactions	id	transaction_id	variant_id → variants.id, CASCADE	transaction_id, variant_id, transaction_type, quantity
tbl_activity_logs	id	log_id	Không có FK tới User	log_id, user_email, action, created_at

Migration không khai báo CHECK. Các ràng buộc NOT NULL của DB yếu hơn một số annotation JPA hoặc yêu cầu nghiệp vụ: ví dụ tbl_users cho phép NULL ở email/password/role; tbl_items cho phép NULL ở name/price; tbl_order_items chỉ bắt buộc id; tbl_orders chỉ bắt buộc id/order_code. Đây là mô tả DDL hiện tại, không phải khuyến nghị cho dữ liệu mới. Unique index trên promotions.code hoặc category.name nullable chỉ duy nhất với các giá trị khác NULL.

Các hành động ON DELETE được khai báo rõ trong V1 gồm: category→item RESTRICT; item→variant CASCADE; modifier group→modifier CASCADE; item/group→bảng nối CASCADE; order→order item CASCADE; variant→inventory transaction CASCADE. Không FK nào khai báo ON UPDATE; MySQL InnoDB áp dụng mặc định NO ACTION, tương đương RESTRICT tức chặn thay đổi khóa cha khi có dòng con tham chiếu [5]. Cascade xóa transaction lịch sử khi xóa Variant là lựa chọn đang có, dù có thể làm giảm khả năng giữ sổ cái lâu dài.

4.3 Đánh giá chuẩn hóa lược đồ hiện tại

Phân tích dưới đây xét lược đồ quan hệ theo dữ liệu lưu, không chỉ theo cách đặt tên entity. Khóa chính surrogate giúp định danh hàng; các thuộc tính mô tả chính không cho thấy phụ thuộc bộ phận trên khóa ghép ngoài bảng nối. Tuy nhiên, thiết kế vẫn có các vùng phi chuẩn hóa hoặc thiếu ràng buộc toàn vẹn cần ghi nhận:

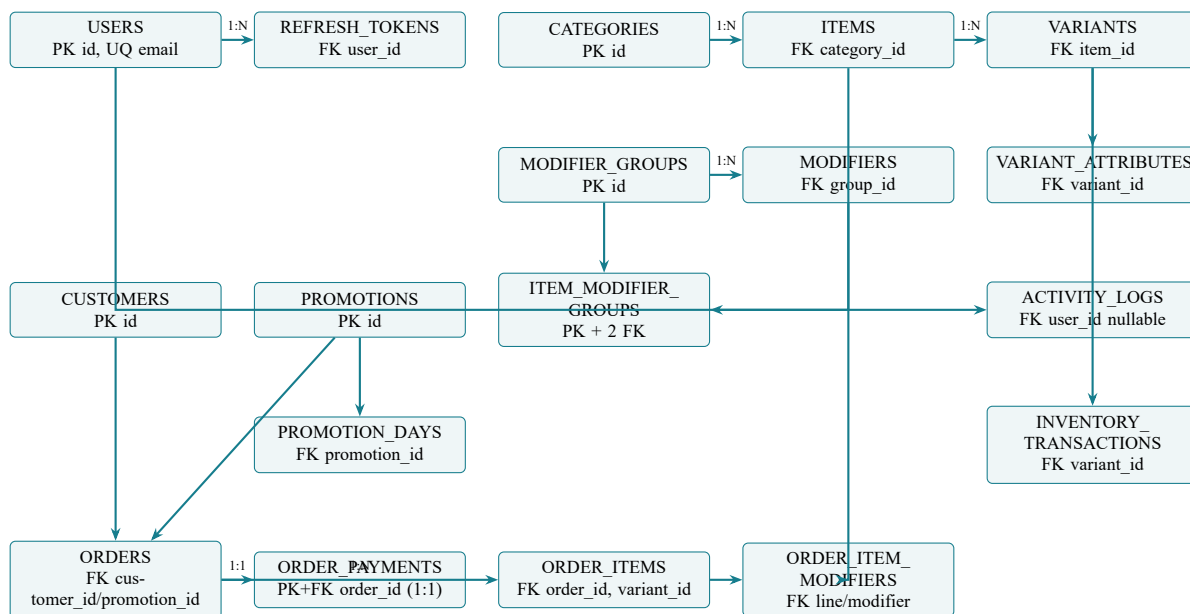
1. **Giá trị nhiều phần/không nguyên tử:** `selected_modifiers` TEXT gộp danh sách modifier của một dòng bán; `days_of_week` được service tách bằng dấu phẩy. Hai trường này biểu diễn tập phần tử trong một ô, nên không đạt 1NF dạng quan hệ chuẩn.
2. **Thuộc tính dẫn xuất lưu lặp:** `variants.cached_stock_quantity` được tính từ tổng ledger; `customers.total_spent/order_count` được cập nhật từ vòng đời đơn; `promotions.times_used` là bộ đếm lượt sử dụng. Các giá trị này là projection/cache, cần đồng bộ với dữ liệu giao dịch gốc.
3. **Thiếu khóa ngoại:** `order.customerId/promotionId`, `orderItem.itemId/variantId`, `refreshToken.userEmail` và `activityLog.userEmail` là scalar không được FK bảo vệ. Dữ liệu mô côi có thể được ghi nếu kiểm tra application thiếu hoặc có tác vụ DB ngoài ứng dụng.
4. **Snapshot giao dịch có chủ đích:** tên, giá, mã khách hàng/khuyến mãi tại thời điểm bán được sao chép vào `order/order item`. Các snapshot này có giá trị lịch sử và không nên bị nối ngược sang dữ liệu danh mục hiện tại để tính lại hóa đơn. Cần phân biệt snapshot nghiệp vụ với trùng lặp thiếu chủ ý.
5. **Ràng buộc miền chưa được DB thực thi:** Không có CHECK cho số lượng dòng, giá, trạng thái, discount, thứ tự ngày; nhiều cột nghiệp vụ nullable dù service mong đợi dữ liệu đầy đủ.

Vì các điểm trên, kết luận phù hợp là: migration đang triển khai có các quan hệ và bảng nối đã chuẩn hóa một phần, nhưng không đủ căn cứ để khẳng định toàn bộ schema đạt 3NF nghiêm ngặt. Đặc biệt, 1NF chưa được bảo đảm với cột đa trị/serialized; do đó tuyên bố 3NF sẽ không chính xác theo định nghĩa quan hệ truyền thống.

4.4 ERD 3NF đề xuất — tách khỏi hiện trạng

Sơ đồ sau là phương án thiết kế học thuật; các bảng/khóa mới **chưa được triển khai trong V1**. Nó đưa mọi giá trị lặp/nhiều phần thành quan hệ riêng; thêm FK cho business associations; tách payment khỏi Order để biểu diễn 1:1; giữ snapshot giá/tên trong đơn như giá trị giao dịch bất biến. Stock/CRM/promotion counters được xem là view hoặc projection, không phải cột chuẩn hóa của bảng gốc.

Mã Mermaid đề xuất, gồm 1:1, 1:N và N:N qua bảng nối:



Hình 4.2: ERD 3NF đề xuất — thiết kế mục tiêu, chưa triển khai

Listing 4.2: ERD mục tiêu chuẩn hóa — không đồng nhất với migration hiện hành

```

1 erDiagram
2     USERS ||--o{ REFRESH_TOKENS : owns
3     USERS o|--o{ ACTIVITY_LOGS : performs
4     CATEGORIES ||--o{ ITEMS : classifies
5     ITEMS ||--o{ VARIANTS : defines
6     VARIANTS ||--o{ VARIANT_ATTRIBUTES : has_atomic_attributes
7     ITEMS ||--o{ ITEM_MODIFIER_GROUPS : links
8     MODIFIER_GROUPS ||--o{ ITEM_MODIFIER_GROUPS : links
9     MODIFIER_GROUPS ||--o{ MODIFIERS : contains
10    PROMOTIONS ||--o{ PROMOTION_DAYS : applies_on
11    CUSTOMERS o|--o{ ORDERS : associated_customer
12    PROMOTIONS o|--o{ ORDERS : applied_promotion
13    ORDERS ||--|| ORDER_PAYMENTS : payment_details
14    ORDERS ||--o{ ORDER_ITEMS : contains
15    VARIANTS ||--o{ ORDER_ITEMS : sold_variant
16    ORDER_ITEMS ||--o{ ORDER_ITEM_MODIFIERS : snapshots
17    MODIFIERS ||--o{ ORDER_ITEM_MODIFIERS : selected_modifier
18    VARIANTS ||--o{ INVENTORY_TRANSACTIONS : ledger
19
20    USERS {
21        bigint id PK
22        varchar email UK
23    }
24    REFRESH_TOKENS {
25        bigint id PK
26        bigint user_id FK
27        varchar token_hash UK
28        varchar family_id
29    }
30    CATEGORIES {
31        bigint id PK
32        varchar category_id UK
33    }
34    ITEMS {

```

```
35     bigint id PK
36     bigint category_id FK
37     varchar item_id UK
38 }
39 VARIANTS {
40     bigint id PK
41     bigint item_id FK
42     varchar sku UK
43     decimal base_price
44 }
45 VARIANT_ATTRIBUTES {
46     bigint variant_id PK, FK
47     varchar attribute_name PK
48     varchar attribute_value
49 }
50 MODIFIER_GROUPS {
51     bigint id PK
52     varchar group_id UK
53 }
54 MODIFIERS {
55     bigint id PK
56     bigint group_id FK
57     varchar modifier_id UK
58 }
59 ITEM_MODIFIER_GROUPS {
60     bigint item_id PK, FK
61     bigint modifier_group_id PK, FK
62 }
63 CUSTOMERS {
64     bigint id PK
65     varchar customer_id UK
66     varchar phone_number UK
67 }
68 PROMOTIONS {
69     bigint id PK
70     varchar promotion_id UK
71     varchar code UK
72 }
73 PROMOTION_DAYS {
74     bigint promotion_id PK, FK
75     tinyint day_of_week PK
76 }
77 ORDERS {
78     bigint id PK
79     bigint customer_id FK "nullable"
80     bigint promotion_id FK "nullable"
81     varchar order_code UK
82 }
83 ORDER_PAYMENTS {
84     bigint order_id PK, FK
85     varchar method
86     varchar status
87     varchar payment_link_id
88 }
89 ORDER_ITEMS {
90     bigint id PK
91     bigint order_id FK
92     bigint variant_id FK
93     int quantity
```

```

94     decimal unit_price_snapshot
95 }
96 ORDER_ITEM_MODIFIERS {
97     bigint id PK
98     bigint order_item_id FK
99     bigint modifier_id FK
100    decimal price_snapshot
101 }
102 INVENTORY_TRANSACTIONS {
103     bigint id PK
104     bigint variant_id FK
105     varchar transaction_type
106     int signed_quantity
107 }
108 ACTIVITY_LOGS {
109     bigint id PK
110     bigint user_id FK "nullable"
111     varchar user_email_snapshot
112 }

```

Trong phương án này, Item–ModifierGroup vẫn là N:N nhưng được phân rã bởi bảng nối có khóa chính ghép. Order–OrderPayment là 1:1 thông qua việc cột `order_id` đồng thời là PK và FK. Order–OrderItem là 1:N; Order–Customer/Promotion là tùy chọn ở phía Order. Variant–Attribute và Promotion–Day tách các tập giá trị khỏi ô text. OrderItem–Modifier được phân rã bằng bảng nối có cột snapshot giá để không phụ thuộc giá Modifier đang có tại thời điểm đọc lịch sử.

4.4.1 Ràng buộc toàn vẹn nên khai báo trong thiết kế mục tiêu

Các quy tắc sau là đề xuất cho migration tương lai. Chúng không xuất hiện như CHECK trong V1, do đó không được gắn nhãn đã cường chế ở database.

Bảng 4.2: Danh mục NOT NULL/UNIQUE/CHECK đề xuất

Phân hệ	Ràng buộc đề xuất	Quy tắc	Trạng thái hiện tại
Tài khoản	NOT NULL + UNIQUE	email, password hash, role; email là candidate key khi NOT NULL. user_id có thể là alternate key nếu bắt buộc nhập.	Email/user_id nullable trong V1.
Refresh token	NOT NULL + UNIQUE	token_hash duy nhất; user_id FK; expires_at lớn hơn issued_at; family_id NOT NULL.	Hash/family/time bắt buộc; chưa có FK user.
Danh mục	NOT NULL + UNIQUE	category_id/name bắt buộc nếu mỗi danh mục phải định danh duy nhất; unique key trên name theo quy tắc cửa hàng.	Các cột nullable.
Biến thể	CHECK + FK	base_price > 0; cached stock không ràng buộc không âm vì cho phép tồn âm; item_id bắt buộc.	NOT NULL/FK một phần; chưa CHECK.
ModifierGroup	CHECK	min_selections ≥ 0; max_selections ≥ min_selections.	NOT NULL; không CHECK.

Phân hệ	Ràng buộc đề xuất	Quy tắc	Trạng thái hiện tại
Order item	NOT NULL + CHECK + FK	order_id/variant_id/quantity bắt buộc; quantity > 0; unit_price_snapshot ≥ 0.	order_id/quantity/IDs nullable; không CHECK.
Payment	NOT NULL + CHECK	order_id unique/PK; method thuộc CASH/PAYOS; status thuộc một trong bốn giá trị PENDING, COMPLETED, FAILED, CANCELLED.	Embedded; status ordinal TINYINT; không CHECK.
Khuyến mãi	CHECK	discount value/min order/max discount không âm; ngày bắt đầu không sau ngày kết thúc; discount percent ≤ 100; BOGO percent thuộc [0,100].	Không CHECK.
Kho	CHECK theo loại	IN có signed quantity > 0; OUT < 0; ADJUSTMENT có thể âm/bằng 0/dương.	Không CHECK; service ký dấu ở application.
CRM	Dẫn xuất	total_spent/order_count nên là view/projection; nếu giữ cache thì giá trị không âm và cập nhật giao dịch nguyên tử.	Cột lưu trực tiếp; không CHECK.
Promotion days/attributes	PK ghép + FK	(promotion_id, day); (variant_id, attribute_name) duy nhất để tách tập giá trị.	Cả hai dùng scalar/string hiện tại.

4.4.2 ON DELETE/ON UPDATE cho quan hệ đề xuất

Khóa chính surrogate được xem là bất biến; mọi FK mục tiêu nên khai báo ON UPDATE RESTRICT/NO ACTION. Với InnoDB, khi bỏ qua ON UPDATE, NO ACTION/RESTRICT được dùng mặc định [5]. Khuyến nghị delete policy giữ bản ghi có giá trị kiểm toán, đồng thời giao dịch xóa chỉ qua service được ủy quyền:

4.5 Ma trận khoảng cách giữa schema hiện tại và thiết kế 3NF

Bảng 4.3: Chính sách tham chiếu đề xuất

Quan hệ FK	ON DELETE đề xuất	ON UPDATE	Lý do
Category → Item	RESTRICT	RESTRICT	Không bỏ danh mục đang phân loại sản phẩm.
Item → Variant	RESTRICT delete hoặc soft	RESTRICT	Không xóa SKU đang được dùng bởi lịch sử bán/kho.
Item/ModifierGroup → bảng nối	CASCADE	RESTRICT	Xóa liên kết thuần cấu hình; không làm mất dòng giao dịch đã snapshot.
ModifierGroup → Modifier	RESTRICT/soft delete	RESTRICT	Modifier cũ có thể được tham chiếu từ OrderItemModifier.
Customer/Promotion → Order	SET NULL	RESTRICT	Giữ hóa đơn và snapshot nếu hồ sơ chủ bị xóa; FK tùy chọn.
Order → OrderPayment	CASCADE	RESTRICT	Quan hệ 1:1 phụ thuộc vòng đời đơn; xóa đơn được kiểm soát ở service.
Order → OrderItem → OrderItemModifier	CASCADE	RESTRICT	Dọn cấu trúc con khi xóa đơn đủ điều kiện; không áp dụng cho đơn COMPLETED.
Variant → Inventory-Transaction	RESTRICT	RESTRICT	Không xóa ledger lịch sử khi SKU bị xóa; ưu tiên archive/soft delete.
User → RefreshToken	CASCADE	RESTRICT	Xóa tài khoản phải thu hồi các phiên liên quan.
User → ActivityLog	SET NULL	RESTRICT	Giữ audit record; lưu email snapshot để còn dấu vết.

Bảng 4.4: Các thay đổi dữ liệu cần có nếu chuyển từ V1 sang mô hình 3NF đề xuất

Hiện trạng V1	Thiết kế mục tiêu	Ảnh hưởng tới ứng dụng
selected_modifiers là TEXT trên tbl_order_items	Tạo ORDER_ITEM_MODIFIERS, lưu mỗi lựa chọn thành một dòng cùng snapshot tên/giá	Cập nhật mapper/DTO và truy vấn chi tiết đơn; giữ khả năng tái hiện giá cũ.
days_of_week là chuỗi phân tách dấu phẩy	Tạo PROMOTION_DAYS một dòng cho mỗi thứ áp dụng	Thay logic split chuỗi bằng query/collection quan hệ.
attributes là JSON map trong TEXT	Tạo VARIANT_ATTRIBUTES với khóa ghép variant/name	Làm tăng số dòng nhưng tạo truy vấn ràng buộc kiểu thuộc tính rõ ràng.
Order.customerId / promotionId scalar	Thêm FK customer_id/promotion_id và giữ snapshot lịch sử	Chính sách xóa phải dùng SET NULL/soft delete để không mất hóa đơn.
OrderItem.itemId/variantId scalar	OrderItem gắn FK đến Variant; tên/giá bán giữ dạng snapshot	Thống nhất Variant là đơn vị bán/tồn; cần migration dữ liệu legacy.
PaymentDetails nhúng trong Order	Tách ORDER_PAYMENTS order_id vừa PK vừa FK	OR- với Tạo quan hệ bảng 1:1, để mở rộng attempt/transaction sau này.
stock/CRM/promotion counters lưu trực tiếp	Tính bằng view hoặc read projection rõ ràng	Cần cân đối hiệu năng với độ nhất quán; nếu giữ cache phải có rebuild/monitor.

KẾT LUẬN VÀ GIỚI HẠN ĐẶC TẢ

Báo cáo đã mô tả actor, biên hệ thống, các use case chính, mô hình miền, lớp nghiệp vụ trọng tâm và quan hệ dữ liệu theo code Billing-app khảo sát. Đặc điểm thiết kế nổi bật là quy trình xử lý đơn QR xuyên qua PayOS; ledger có signed quantity là nguồn tính tồn; cancellation là một thao tác bù trừ nhiều phân hệ; refresh token được xoay vòng theo family; và một số thông tin khách hàng/khuyến mãi được chụp thành snapshot trong đơn.

Kết quả phân tích cũng cho thấy cần phân biệt invariant ứng dụng với ràng buộc DB. Việc chặn xóa đơn COMPLETED nằm ở service, không nằm trong CHECK/trigger; nhiều trường trong bảng có thể NULL; một số mã tham chiếu không có FK; migration không có CHECK; cột modifier/ngày áp dụng chứa cấu trúc nhiều phần; và một số trường tồn/CRM/promotion là cache. Do đó ERD triển khai không được mô tả là 3NF nghiêm ngặt. ERD 3NF riêng đưa ra hướng tách giá trị đa trị, khóa ngoại và payment 1:1, song đây là thiết kế mục tiêu chứ chưa phải trạng thái đã cài đặt.

Những nội dung chưa được xác nhận gồm: chỉ tiêu hiệu năng/availability, khả năng offline đồng bộ đầy đủ, chính sách lưu trữ lịch sử lâu dài, mức idempotency của webhook, chiến lược xử lý payment callback trễ và bảo đảm commit việc thu hồi token family khi replay. Báo cáo không gán kết quả kiểm thử pass/fail; đây là đặc tả được suy ra từ cấu hình và source đã khảo sát, không phải biên bản kiểm thử nghiệm thu.

Tài liệu tham khảo

- [1] ISO/IEC/IEEE, *ISO/IEC/IEEE 29148:2018 — Systems and software engineering: Life cycle processes — Requirements engineering*, 2nd ed., 2018. Bản này được ISO rà soát/duy trì hiệu lực năm 2024. [Trực tuyến] <https://www.iso.org/standard/72089.html>.
- [2] Object Management Group, *OMG Unified Modeling Language (OMG UML), Version 2.5.1*, Dec. 2017. [Trực tuyến] <https://www.omg.org/spec/UML/2.5.1/About-UML>.
- [3] Mermaid, *Class diagrams syntax*. [Trực tuyến] <https://mermaid.js.org/syntax/classDiagram.html>.
- [4] Mermaid, *Entity Relationship Diagrams syntax*. [Trực tuyến] <https://mermaid.js.org/syntax/entityRelationshipDiagram.html>.
- [5] Oracle, *MySQL 8.4 Reference Manual, Foreign Key Constraints*. [Trực tuyến] <https://dev.mysql.com/doc/refman/8.4/en/create-table-foreign-keys.html>.
- [6] Oracle, *MySQL 8.4 Reference Manual, CHECK Constraints*. [Trực tuyến] <https://dev.mysql.com/doc/refman/8.4/en/create-table-check-constraints.html>.
- [7] Spring Framework, *@Transactional API and rollback defaults*. [Trực tuyến] <https://docs.spring.io/spring-framework/docs/current/javadoc-api/org.springframework.transaction.annotation.Transactional.html>.
- [8] Billing-app, *CONTEXT.md*, tài liệu tổng quan hệ thống trong repository, khảo sát ngày 24/09/2026.
- [9] Billing-app, *billingsoftware/src/main/resources/db/migration/V1__init_schema.sql*, migration Flyway, khảo sát ngày 24/09/2026.
- [10] Billing-app, *docs/adr/0001–0009*, quyết định kiến trúc, đối chiếu với mã triển khai ngày 24/09/2026.
- [11] Billing-app, *OrderController.java; OrderServiceImpl.java; OrderEntity.java; OrderItemEntity.java; PaymentWebhookController.java*, mã nguồn backend.
- [12] Billing-app, *InventoryController.java; InventoryServiceImpl.java; InventoryTransactionEntity.java; VariantEntity.java*, mã nguồn backend.
- [13] Billing-app, *AuthController.java; RefreshTokenService.java; RefreshTokenEntity.java; SecurityConfig.java*, mã nguồn backend.
- [14] Billing-app, *PromotionServiceImpl.java; PromotionEntity.java; CustomerEntity.java; ActivityLogServiceImpl.java*, mã nguồn backend.